

ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA TOÁN - CƠ - TIN HỌC

TỐI ƯU HÓA NÂNG CAO

BÁO CÁO

**Đề tài: Ứng dụng các thuật toán tối ưu hóa bậc một và bậc hai
trong bài toán hồi quy tuyến tính có hiệu chỉnh Ridge
định giá điện thoại đã qua sử dụng**



Học viên thực hiện:

Nguyễn Hồng Phúc (25007860)

Phạm Huy Hoàng (25007878)

Bùi Quang Huy (25007877)

Ngày 10 tháng 8 năm 2026

Tóm tắt

Nhóm tự cài đặt bốn thuật toán tối ưu hóa bất buộc cùng ba phương pháp mở rộng, rồi so sánh chúng trên một bài toán hồi quy tuyến tính có hiệu chỉnh Ridge với dữ liệu định giá điện thoại đã qua sử dụng. Hàm mục tiêu cố định cho mọi thí nghiệm là hàm bậc hai lồi mạnh nên nó có nghiệm đóng, và f^* tính được trước khi chạy bất kỳ thuật toán nào; nhờ vậy sai số $f(w_k) - f^*$ đo được tuyệt đối chứ không phải so với giá trị tốt nhất quan sát được, còn hệ số co rút thực nghiệm thì đối chiếu thẳng với cận lý thuyết. Báo cáo gồm chín chương, trong đó bốn chương thí nghiệm lần lượt khảo sát ảnh hưởng của tham số tối từng thuật toán, so sánh các thuật toán ở cấu hình tốt nhất, đối chiếu với cài đặt mặc định của scikit-learn, và phân tích vai trò của hệ số hiệu chỉnh cùng việc chuẩn hóa dữ liệu đối với số điều kiện của bài toán.

Mục lục

Tóm tắt	1
I Đặt vấn đề và phát biểu bài toán	5
1.1 Dữ liệu	5
1.2 Hàm mục tiêu	5
1.3 Các hằng số chi phối tốc độ hội tụ	6
II Chuẩn bị dữ liệu	8
2.1 Tạo biến tương tác trên cột đã chuẩn hóa	8
2.2 Chuẩn hóa cột và số điều kiện	8
2.3 Chọn hệ số hiệu chỉnh	10
III Các thuật toán đã cài đặt	12
3.1 Gradient descent	12
3.2 Backtracking theo điều kiện Armijo	12
3.3 Stochastic gradient descent	12
3.4 Accelerated gradient descent	13
3.5 Phương pháp Newton	13

3.6	Các thuật toán mở rộng	13
IV	Kiểm thử tính đúng đắn của cài đặt	15
V	Khảo sát tham số của từng thuật toán	16
5.1	Gradient descent với độ dài bước cố định	16
5.2	Gradient descent với backtracking	19
5.3	SGD: kích thước lô	20
5.4	SGD: quy tắc chọn độ dài bước	22
5.5	Accelerated gradient descent	24
5.6	Phương pháp Newton	26
5.7	L-BFGS với các kích thước bộ nhớ	28
VI	So sánh tổng hợp	30
6.1	Đối chiếu với tốc độ lý thuyết	30
VII	So sánh với thư viện	34
VIII	Ảnh hưởng của hệ số hiệu chỉnh	37
IX	Kết luận	40
	Tài liệu tham khảo	42

Danh sách hình vẽ

2.1	Ảnh hưởng của việc chuẩn hóa cột lên tốc độ hội tụ của gradient descent, theo số vòng lặp. Trục tung là $f(w_k) - f^*$ trên thang logarit.	9
2.2	Ảnh hưởng của việc chuẩn hóa cột lên tốc độ hội tụ của gradient descent, theo thời gian chạy.	9
2.3	Sai số cross-validation 5 fold theo λ . Dải xám là một sai số chuẩn quanh giá trị nhỏ nhất, đường chấm là điểm cực tiểu và đường gạch là giá trị được chọn.	10
5.1	Gradient descent với các độ dài bước cố định, theo số vòng lặp.	17
5.2	Gradient descent với các độ dài bước cố định, theo thời gian chạy.	17
5.3	Ảnh hưởng của α và β , theo số vòng lặp.	18
5.4	Ảnh hưởng của α và β , theo thời gian chạy.	18
5.5	SGD với các kích thước lô khác nhau, theo số lượt duyệt dữ liệu.	20
5.6	SGD với các kích thước lô khác nhau, theo thời gian chạy.	21
5.7	Cùng một độ dài bước $0.1/L$ cho mọi kích thước lô. Lô kích thước 1 phân kỳ.	21
5.8	Cùng một độ dài bước $0.1/L$, theo thời gian chạy.	22
5.9	Các quy tắc chọn η_k , theo số lượt duyệt dữ liệu.	23
5.10	Các quy tắc chọn η_k , theo thời gian chạy.	23
5.11	Các cấu hình momentum, có và không khởi động lại, theo số vòng lặp.	24
5.12	Các cấu hình momentum, có và không khởi động lại, theo thời gian chạy.	25
5.13	Các biến thể Newton, theo số vòng lặp.	27
5.14	Các biến thể Newton, theo thời gian chạy.	27
5.15	L-BFGS với các kích thước bộ nhớ m , theo số vòng lặp.	28
5.16	L-BFGS với các kích thước bộ nhớ m , theo thời gian chạy.	29
6.1	So sánh các thuật toán ở cấu hình tốt nhất, theo số vòng lặp. Trục hoành cắt ở 200 vòng lặp cho dễ đọc; gradient descent với bước cố định chạy tiếp theo đúng đường thẳng đó tới vòng lặp 1200.	31
6.2	So sánh các thuật toán ở cấu hình tốt nhất, theo thời gian chạy. Trục hoành cắt ở 6 giây; gradient descent với bước cố định chạy tiếp tới 44,3 s.	31
6.3	Đường hội tụ thực nghiệm đặt cạnh hai cận lý thuyết.	32

7.1	So sánh các phương pháp lặp tự cài đặt, theo số vòng lặp.	35
7.2	So sánh mã tự viết với thư viện, theo thời gian chạy. Các cài đặt thư viện không có lịch sử vòng lặp nên hiện ra thành một điểm trên mặt phẳng (thời gian, sai số).	35
8.1	Tốc độ hội tụ ứng với các giá trị λ khác nhau, theo số vòng lặp. Cả ba lần chạy dùng gradient descent với $t = 1/L$	38
8.2	Tốc độ hội tụ ứng với các giá trị λ khác nhau, theo thời gian chạy.	38

Danh sách bảng

1.1	Thành phần 28 cột của bộ dữ liệu gốc.	6
1.2	Các hằng số của bài toán sau khi cố định dữ liệu và λ	6
5.1	Gradient descent với độ dài bước cố định. Ngưỡng ổn định là $2/L = 0,745$	19
5.2	Chi phí của line search, đo trong 100 vòng lặp đầu. Ba cột giữa là số lần đánh giá hàm mục tiêu trên mỗi vòng lặp, ứng với ba giá trị của α	19
5.3	Sai số $f - f^*$ sau 30 lượt duyệt dữ liệu, $B = 64$	22
5.4	Các cấu hình của phương pháp tăng tốc.	25
5.5	Các biến thể của phương pháp bậc hai.	26
5.6	L-BFGS với bốn kích thước bộ nhớ. Hai cột cuối đo tới khi thuật toán chạm giới hạn số học, không dừng ở ngưỡng 10^{-6}	28
6.1	Tổng hợp kết quả của từng thuật toán ở cấu hình tốt nhất. Cột <i>tổng thời gian</i> là thời gian chạy tới khi thuật toán không cải thiện được nữa, tức tới giới hạn số học.	32
6.2	Hệ số co rút mỗi vòng lặp, quan sát được so với cận lý thuyết.	32
7.1	Cài đặt mặc định của scikit-learn, đánh giá bằng hàm mục tiêu (1.1) của bài toán. Cột <i>sai số</i> là $f(\hat{w}) - f^*$	36
8.1	Hệ số hiệu chỉnh vừa quyết định số điều kiện, vừa quyết định chất lượng dự báo. Cả ba lần chạy dùng gradient descent với $t = 1/L$, nên cột số vòng lặp đặt cạnh được với dòng $t = 1/L$ của bảng 5.1.	39

I. Đặt vấn đề và phát biểu bài toán

1.1 Dữ liệu

Bộ dữ liệu *Used Phone Price Prediction* trên Kaggle gồm 1 000 000 bản ghi và 28 cột, phân bố theo bảng 1.1, trong đó không cột nào có giá trị thiếu và không bản ghi nào trùng lặp. Vì bản đăng trên Kaggle đã qua tiền xử lý, phần chuẩn bị dữ liệu ở chương II bỏ được cả bước điền khuyết lẫn bước khử trùng lặp, và chỉ còn mã hóa biến định tính rồi chuẩn hóa cột. Thuận lợi này gắn với nguồn dữ liệu chứ không gắn với bài toán: nếu thu thập trực tiếp từ các tin rao bán thì hai con số 0 nói trên không còn, và nhóm dự đoán khi đó chi phí làm sạch sẽ vượt hẳn chi phí của phần tối ưu hóa.

Giá bán lại lệch phải rõ rệt, với trung vị 18 555 thấp hơn hẳn trung bình 22 598 trong khi giá trị lớn nhất lên tới 160 257, nên nhóm lấy logarit biến mục tiêu trước khi hồi quy. Giữ nguyên thang gốc thì bình phương sai số để vài trăm bản ghi đắt tiền chi phối hàm mục tiêu, và nghiệm thu được sẽ phản ánh riêng nhóm đó. Đổi lại, mọi giá trị RMSE trong báo cáo đọc trên thang logarit nên không so trực tiếp với giá bằng đơn vị tiền được.

Toàn bộ thí nghiệm chạy trên một mẫu ngẫu nhiên 200 000 bản ghi, chia thành 160 000 điểm huấn luyện và 40 000 điểm kiểm tra. Nhóm lấy mẫu vì chi phí tính toán chứ không vì lý do thống kê: trên toàn bộ một triệu bản ghi, một lần tính gradient mất 271 ms thay vì 39 ms, và vì lưới tham số ở chương V nhân con số đó lên theo tổng số vòng lặp của mọi cấu hình nên chênh lệch gần bảy lần ấy nhân lên theo toàn bộ lưới. Cỡ mẫu này đủ cho các kết luận về tốc độ hội tụ, vốn chỉ phụ thuộc L , μ và κ , nhưng sẽ không đủ nếu đại lượng cần đo phụ thuộc trực tiếp vào n , chẳng hạn phương sai gradient của SGD ở mục 5.3.

Tỉ lệ chia 160 000 trên 40 000 không chạm tới phần tối ưu hóa, vì f^* ở (1.3) tính trên tập huấn luyện còn tập kiểm tra chỉ vào RMSE. Chỉ khi nào λ được chọn bằng tập kiểm tra thay vì bằng cross-validation như ở mục 2.3 thì tỉ lệ ấy mới ảnh hưởng.

1.2 Hàm mục tiêu

Ký hiệu n là số điểm dữ liệu, d là số thuộc tính sau khi mã hóa, $X \in \mathbb{R}^{n \times d}$ là ma trận thiết kế và $y \in \mathbb{R}^n$ là vector mục tiêu. Bài toán tối ưu hóa xét ở đây là

$$\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2. \quad (1.1)$$

Bảng 1.1: Thành phần 28 cột của bộ dữ liệu gốc.

Nhóm cột	Số cột	Ví dụ
Định lượng liên tục	14	dung lượng RAM, tuổi máy, giá gốc, điểm camera
Định lượng nhị phân	7	screen_cracked, box_available
Định tính	6	brand 7 mức, model 23 mức
Biến mục tiêu	1	resale_price

Bảng 1.2: Các hằng số của bài toán sau khi cố định dữ liệu và λ .

Đại lượng	Ý nghĩa	Giá trị
n	số điểm huấn luyện	160 000
d	số thuộc tính sau mã hóa	280
λ	hệ số hiệu chỉnh	0,01
L	hằng số Lipschitz	2,683 09
μ	hệ số lỗi mạnh	0,01
κ	số điều kiện	268,309
f^*	giá trị tối ưu	0,023 743 03
$\ w^*\ $	chuẩn của nghiệm	0,781 434

Nhóm xử lý hệ số chặn bằng cách chuẩn hóa X theo cột và trừ trung bình khỏi y , nên bài toán chỉ còn một khối biến w và không tọa độ nào nằm ngoài thành phần hiệu chỉnh. Cách khác là giữ lại một cột hằng làm hệ số chặn, nhưng khi đó tọa độ ấy cũng chịu phạt λ và nghiệm lệch khỏi mô hình có chặn tự do.

Gradient và Hessian của (1.1) là

$$\nabla f(w) = \frac{1}{n}X^\top(Xw - y) + \lambda w, \quad \nabla^2 f(w) = \frac{1}{n}X^\top X + \lambda I, \quad (1.2)$$

trong đó Hessian không phụ thuộc w , nên bài toán lỗi mạnh và có nghiệm đóng

$$w^* = \left(\frac{1}{n}X^\top X + \lambda I\right)^{-1} \left(\frac{1}{n}X^\top y\right), \quad f^* = f(w^*). \quad (1.3)$$

1.3 Các hằng số chi phối tốc độ hội tụ

$$L = \lambda_{\max}\left(\frac{1}{n}X^\top X\right) + \lambda, \quad \mu = \lambda_{\min}\left(\frac{1}{n}X^\top X\right) + \lambda, \quad \kappa = \frac{L}{\mu}. \quad (1.4)$$

Mã trận Gram $\frac{1}{n}X^\top X$ có hạng 273 trên 280, và ngay cả khi bỏ qua 7 hướng suy biến đó thì trị riêng kế tiếp cũng chỉ cỡ $2 \cdot 10^{-8}$, tức nhỏ hơn tám bậc so với $L = 2,683 09$ ở bảng 1.2. Phần đóng góp của dữ liệu vào μ vì thế coi như bằng không, và

$$\mu = \lambda, \quad \kappa = \frac{L}{\lambda}. \quad (1.5)$$

Hệ số hiệu chỉnh do đó là đại lượng duy nhất quyết định số điều kiện của bài toán, còn cấu trúc dữ liệu thì không. Quan hệ này chỉ giữ khi ma trận Gram còn suy biến; nếu mọi cột độc lập tuyến tính và trị riêng nhỏ nhất cùng bậc với λ thì μ quay về do dữ liệu quyết định. Chương VIII khai thác tính chất này.

II. Chuẩn bị dữ liệu

Phần xử lý dữ liệu giữ ở mức tối thiểu, vì trọng tâm của bài tập là tối ưu hóa. Nhóm chuyển các cột dạng chuỗi có đơn vị về số, mã hóa one-hot toàn bộ biến định tính, bổ sung tích từng cặp và bình phương của các cột định lượng, rồi chuẩn hóa mọi cột về trung bình 0 và độ lệch chuẩn 1. Các cột hằng bị loại vì không mang thông tin, còn bình phương của 7 cột nhị phân bị loại vì nó tái tạo đúng cột gốc.

Số thuộc tính sau mã hóa là $d = 280$, gồm 21 cột định lượng gốc, 210 tích từng cặp, 14 bình phương và 35 cột one-hot. Phần lớn kích thước bài toán vì thế do biến tương tác sinh ra chứ không do biến định tính, bởi tích từng cặp của 21 cột cho $\binom{21}{2} = 210$ cột, gấp sáu lần toàn bộ phần one-hot của 6 biến định tính với 41 mức ở bảng 1.1. Tỷ lệ này sẽ đảo nếu một biến định tính như model có hàng nghìn mức thay vì 23 mức.

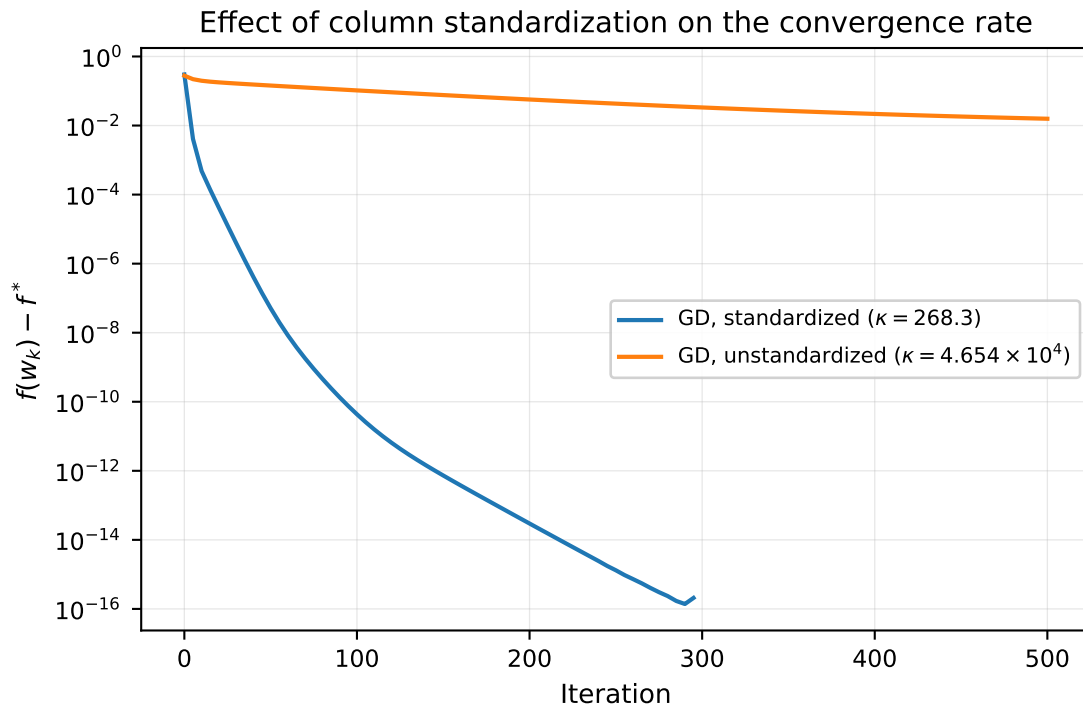
Ba mục sau trình bày ba quyết định có ảnh hưởng trực tiếp tới bài toán tối ưu hóa: cách tạo biến tương tác, cách chuẩn hóa cột, và cách chọn hệ số hiệu chỉnh.

2.1 Tạo biến tương tác trên cột đã chuẩn hóa

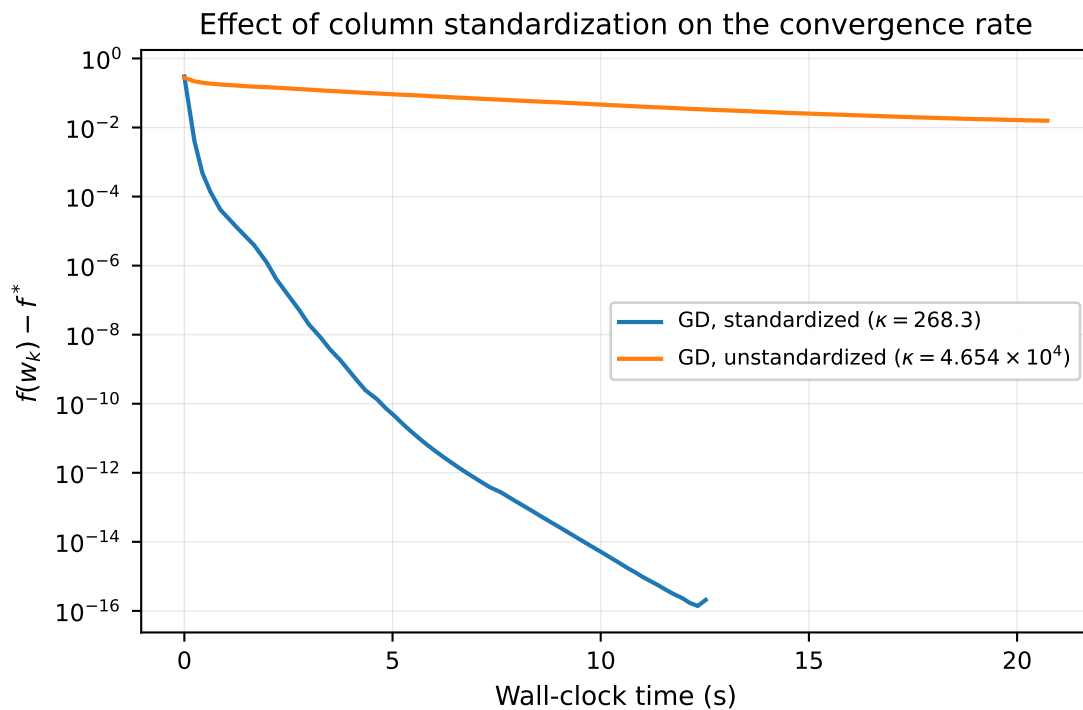
Lấy tích trực tiếp trên thang gốc, tức nhân `original_price` cỡ 10^5 với `screen_size_inches` cỡ 6, thì ma trận thiết kế thu được có $L = 17,14$ và 48 trị riêng dưới 10^{-6} ; lấy tích trên các cột đã chuẩn hóa thì $L = 2,68$ và số trị riêng dưới 10^{-6} trở về đúng 7, bằng mức của khối cột gốc. Chênh lệch 6,4 lần về số điều kiện giữa hai cách hoàn toàn do cách biểu diễn, vì một cột tích trên thang gốc có phương sai lớn hơn các cột còn lại nhiều bậc và kéo riêng trị riêng lớn nhất lên, trong khi không gian cột sinh ra thì vẫn như cũ. Hai cách chỉ trùng nhau khi mọi cột định lượng vốn đã cùng một thang đo.

2.2 Chuẩn hóa cột và số điều kiện

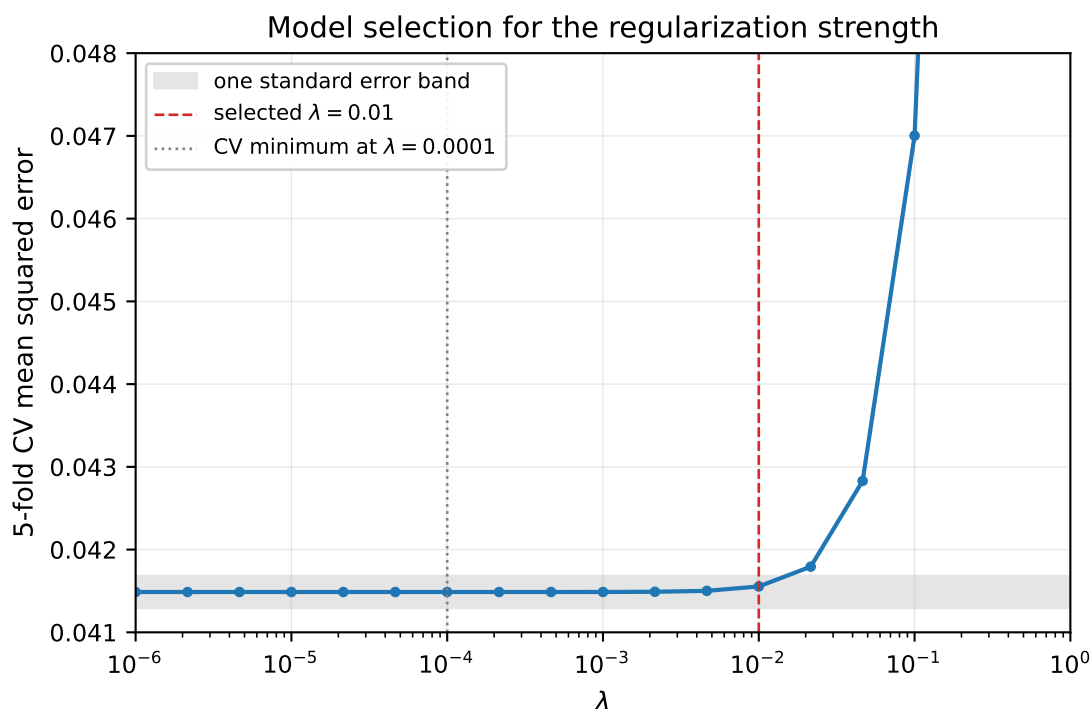
Với cùng dữ liệu và cùng λ , số điều kiện là $\kappa = 268,3$ sau khi chuẩn hóa cột nhưng lên tới $\kappa = 46\,540$ khi các cột giữ thang đo không đồng nhất, tức chênh nhau 173 lần. Chênh lệch đó hiện thẳng ra ở tốc độ hội tụ trên hình 2.1: bản đã chuẩn hóa chạm giới hạn số học 10^{-16} sau khoảng 290 vòng lặp, còn bản giữ thang đo gốc chạy hết 500 vòng mà sai số mới xuống cỡ 10^{-2} , tức vẫn cao hơn mức bản chuẩn hóa đạt được trong mười vòng đầu tiên. Hệ số co rút mỗi vòng lặp của gradient descent có dạng $1 - \mathcal{O}(1/\kappa)$, nên khi κ lớn hơn 173 lần thì phần sai số cắt được sau mỗi vòng cũng nhỏ đi theo đúng tỷ lệ ấy.



Hình 2.1: Ảnh hưởng của việc chuẩn hóa cột lên tốc độ hội tụ của gradient descent, theo số vòng lặp. Trục tung là $f(w_k) - f^*$ trên thang logarit.



Hình 2.2: Ảnh hưởng của việc chuẩn hóa cột lên tốc độ hội tụ của gradient descent, theo thời gian chạy.



Hình 2.3: Sai số cross-validation 5 fold theo λ . Dải xám là một sai số chuẩn quanh giá trị nhỏ nhất, đường chấm là điểm cực tiểu và đường gạch là giá trị được chọn.

Chi phí mỗi vòng lặp không phụ thuộc cách biểu diễn cột, khoảng 42 ms cho cả hai, nên hình 2.2 chỉ là hình 2.1 vẽ lại trên một thang khác, với bản chuẩn hóa xuống tới giới hạn số học trong khoảng 12 giây còn bản kia sau hơn 20 giây vẫn dừng ở 10^{-2} . Chuẩn hóa cột vì thế không phải một bước tiền xử lý về thống kê, mà là bước quyết định bài toán tối ưu hóa có giải được trong thời gian hợp lý hay không. Lợi ích ấy chỉ tính cho các phương pháp bậc một, vì Newton ở mục 5.6 vẫn hội tụ sau đúng một vòng lặp dù κ bằng bao nhiêu.

2.3 Chọn hệ số hiệu chỉnh

Đường cong cross-validation 5 fold ở hình 2.3 cho sai số trung bình giống nhau tới bốn chữ số thập phân với mọi λ từ 10^{-6} đến 10^{-3} , tức phẳng suốt ba bậc thập phân, vì mức hiệu chỉnh nhỏ hơn nhiều của dữ liệu thì không đổi được sai số dự báo. Trong khi đó, theo (1.5), số điều kiện giữa hai đầu khoảng ấy chênh nhau 1000 lần. Vậy λ vẫn phải chọn cẩn thận, nhưng tiêu chí chọn phải nằm ngoài cross-validation; đường cong chỉ có cực tiểu rõ khi n nhỏ hơn nhiều hoặc d gần n , và bài toán này không rơi vào trường hợp đó.

Điểm cực tiểu của một đường cong phẳng như vậy chỉ phản ánh nhiễu của phép chia fold, nên nhóm dùng quy tắc một sai số chuẩn: chọn λ lớn nhất mà sai số cross-validation của nó vẫn nằm trong một sai số chuẩn của giá trị tốt nhất. Quy tắc này cho $\lambda = 0,01$ ứng với $\kappa = 268,3$, trong khi điểm cực tiểu nằm ở $\lambda = 10^{-4}$ ứng với $\kappa = 26\,831$.

Về mặt thống kê, cái giá phải trả không đáng kể: RMSE trên tập kiểm tra là 0,206 01 tại $\lambda = 0,01$ so với 0,205 79 tại điểm cực tiểu cross-validation, kém đi 0,1%. Đổi lại, κ giảm đúng 100 lần theo tỉ lệ giữa hai giá trị λ , và toàn bộ các thuật toán bậc một trở nên so sánh được với nhau trong thời gian chạy hợp lý. Đánh đổi này chỉ có lợi vì đường cong phẳng và vì bài toán giải bằng phương pháp lặp; với một đường cong có cực tiểu rõ thì khoản mất 0,1% kia lớn hơn nhiều, còn với thuật toán bậc hai thì κ không chi phối số vòng lặp nên khoản được cũng biến mất.

III. Các thuật toán đã cài đặt

3.1 Gradient descent

Với hàm bậc hai lồi mạnh, phương pháp hội tụ khi $0 < t < 2/L$ và độ dài bước tối ưu là $t = 2/(L + \mu)$ [1].

3.2 Backtracking theo điều kiện Armijo

Xuất phát từ $t = t_0$, thu nhỏ $t \leftarrow \beta t$ cho tới khi [2]

$$f(w + tp) \leq f(w) + \alpha t \langle \nabla f(w), p \rangle, \quad (3.1)$$

với p là hướng giảm. Khi $p = -\nabla f(w)$, điều kiện trở thành $f(w - t\nabla f(w)) \leq f(w) - \alpha t \|\nabla f(w)\|_2^2$.

3.3 Stochastic gradient descent

$$g_k = \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} (x_i^\top w_k - y_i) x_i + \lambda w_k, \quad w_{k+1} = w_k - \eta_k g_k. \quad (3.2)$$

Nhóm thử bốn quy tắc chọn độ dài bước [3]

$$\eta_k = \eta_0, \quad \eta_k = \frac{\eta_0}{1 + \gamma k}, \quad \eta_k = \frac{\eta_0}{\sqrt{k+1}}, \quad \eta_k = \eta_0 2^{-\lfloor k/p \rfloor}. \quad (3.3)$$

Ba thuật toán còn lại của báo cáo đều có một biến thể dùng backtracking, riêng SGD thì không, vì điều kiện Armijo (3.1) đòi hỏi đánh giá f trên toàn bộ tập huấn luyện ở mỗi lần thử độ dài bước, trong khi với $B = 64$ một lượt duyệt dữ liệu (epoch) gồm 2500 lần cập nhật. Lấy 39 ms làm chi phí một lần quét toàn bộ dữ liệu thì riêng phần line search cần khoảng 98 s cho một lượt duyệt, còn cả lượt duyệt đo được chỉ mất 0,082 s, tức 2,45 s cho 30 lượt. Con số 98 s là ước lượng suy ra từ hai đại lượng đo được và chưa tính số lần thu nhỏ bước trong mỗi lần gọi line search, nên chênh lệch thật chỉ có thể lớn hơn.

Trở ngại thứ hai nằm ở phần bảo đảm. Áp (3.1) lên g_k chỉ bảo đảm giảm cho hàm mất mát của lô hiện tại chứ không cho f , nên một độ dài bước được chấp nhận trên lô này vẫn có thể làm f tăng. Vì vậy độ dài bước ban đầu lấy theo hằng số trơn của lô, $\eta_0 = 1/L_B$, và phần khảo sát tham số của SGD ở chương V so sánh bốn quy

Algorithm 3.1 Gradient descent

Require: objective f , initial point w_0 , step size rule

```

1: for  $k = 0, 1, 2, \dots$  do
2:    $g_k \leftarrow \nabla f(w_k)$ 
3:   choose  $t_k$  (fixed, or by Armijo backtracking)
4:    $w_{k+1} \leftarrow w_k - t_k g_k$ 
5: end for

```

tắc ở (3.3) thay cho việc so sánh các tham số α và β . Bốn quy tắc đó ấn định η_k trước theo chỉ số vòng lặp và không đọc gì từ giá trị hàm mục tiêu, nên chúng là quy tắc tắt định chứ không phải một dạng line search.

3.4 Accelerated gradient descent

$$y_k = w_k + \beta_k(w_k - w_{k-1}), \quad w_{k+1} = y_k - t \nabla f(y_k), \quad (3.4)$$

với $\beta = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ khi biết μ [1], hoặc $\beta_k = (k - 1)/(k + 2)$ khi không [4]. Cơ chế khởi động lại đặt $k = 0$ mỗi khi $\nabla f(y_k)^\top (w_{k+1} - w_k) > 0$ [5].

3.5 Phương pháp Newton

Bước Newton giải hệ $\nabla^2 f(w_k) p_k = -\nabla f(w_k)$ bằng phân rã Cholesky [6]. Với hàm mục tiêu bậc hai và $w_0 = 0$,

$$w_1 = w_0 - (\nabla^2 f)^{-1} \nabla f(w_0) = \left(\frac{1}{n} X^\top X + \lambda I \right)^{-1} \left(\frac{1}{n} X^\top y \right) = w^*, \quad (3.5)$$

nghĩa là phương pháp hội tụ sau đúng một vòng lặp và bước Newton trùng với nghiệm đóng. Phép so sánh có ý nghĩa ở đây do đó không phải là số vòng lặp, mà là chi phí $\mathcal{O}(nd^2 + d^3)$ của một vòng lặp Newton đặt cạnh chi phí $\mathcal{O}(nd)$ của một vòng lặp gradient. Trên bài toán phi tuyến, nơi Hessian thay đổi theo w và mỗi bước Newton chỉ là một xấp xỉ địa phương, số vòng lặp trở lại thành đại lượng đáng so.

3.6 Các thuật toán mở rộng

Ngoài bốn thuật toán bắt buộc, nhóm cài đặt thêm ba phương pháp, mỗi phương pháp làm rõ một khía cạnh khác nhau của chi phí mỗi vòng lặp.

Heavy ball [7]. Cùng dạng momentum như (3.4) nhưng ngoại suy sau khi lấy gradient thay vì trước: $w_{k+1} = w_k - t \nabla f(w_k) + \beta(w_k - w_{k-1})$, với bộ tham số tối ưu cho hàm bậc hai là $t = 4/(\sqrt{L} + \sqrt{\mu})^2$ và $\beta =$

$\left(\frac{(\sqrt{\kappa} - 1)}{(\sqrt{\kappa} + 1)}\right)^2$. Nhóm cài đặt nó để đặt cạnh Nesterov, vì hai công thức chỉ khác nhau ở thứ tự giữa bước ngoại suy và bước lấy gradient.

Newton-CG. Giải hệ Newton bằng gradient liên hợp thay vì phân rã Cholesky. Mỗi bước CG chỉ cần một tích Hessian nhân vector, tính được bằng $\frac{1}{n}X^\top(Xv) + \lambda v$ với chi phí $\mathcal{O}(nd)$, nên không bao giờ phải lập ma trận $d \times d$. Phương án này dùng được cả khi d lớn tới mức $\mathcal{O}(d^3)$ không chấp nhận được.

L-BFGS. Xấp xỉ nghịch đảo Hessian từ m cặp (s_k, y_k) gần nhất qua đệ quy hai vòng [2], chi phí $\mathcal{O}(md)$ mỗi vòng lặp. Phương pháp này nằm giữa nhóm bậc một và nhóm bậc hai về cả chi phí lẫn tốc độ, và là lựa chọn mặc định trong phần lớn thư viện tối ưu hóa tổng quát.

Nhóm cũng cài đặt Adam [8] nhưng không đưa vào so sánh chính, vì trên một bài toán bậc hai lồi mạnh có nghiệm đóng, cơ chế chia theo căn bậc hai của moment bậc hai không mang lại lợi thế nào so với các phương pháp đã có. Cơ chế ấy chỉ phát huy khi thang gradient lệch nhau nhiều giữa các tọa độ, điều mà bước chuẩn hóa cột ở chương II đã loại bỏ.

IV. Kiểm thử tính đúng đắn của cài đặt

Trước khi chạy lưới tham số, 46 phép thử tự động kiểm tra toàn bộ cài đặt. Nhóm đối chiếu được trực tiếp với công thức giải tích vì bài toán có sẵn nghiệm đóng và có sẵn biểu thức của gradient lẫn Hessian; bốn phép thử dưới đây khai thác đúng điều đó.

1. **Gradient giải tích so với sai phân trung tâm.** So sánh $[\nabla f(w)]_i$ với $(f(w + \varepsilon e_i) - f(w - \varepsilon e_i))/(2\varepsilon)$ tại $\varepsilon = 10^{-6}$. Sai số tương đối dưới 10^{-6} .
2. **Hessian so với sai phân của gradient.** Cùng ngưỡng.
3. **Nghiệm đóng là điểm dừng.** $\|\nabla f(w^*)\| < 10^{-12}$.
4. **Mọi thuật toán cùng hội tụ về một nghiệm.** Trên bài toán tổng hợp $n = 100$, $d = 5$, cả mười cấu hình đều cho $\|w_{\text{cuối}} - w^*\| < 10^{-6}$.

Ngoài ra có ba phép thử về mặt phương pháp luận:

- gradient descent phải phân kỳ khi $t > 2/L$;
- SGD với độ dài bước hằng phải dừng lại cách f^* một khoảng dương;
- ghi lịch sử với tần suất dày hay thưa phải cho cùng số lần đánh giá hàm mục tiêu, tức chi phí theo dõi không lẫn vào chi phí thuật toán.

Các phép thử và chính các thí nghiệm đã bắt được hai lỗi có thật chứ không chỉ để trình bày. Lỗi thứ nhất nằm ở cách nhận diện cột số có đơn vị, vốn biến cột model thành số (chương II); lỗi thứ hai là momentum của phương pháp tăng tốc lúc đầu không nhất quán với độ dài bước do line search chọn, và nó đẩy hàm mục tiêu lên $1,47 \times 10^4$ (mục 5.5). Cả hai đều để chương trình chạy trót lọt và trả về một con số trông hợp lý, nên không phép thử nào đối chiếu với mốc giải tích thì chúng còn nguyên trong kết quả.

V. Khảo sát tham số của từng thuật toán

Tất cả số liệu trong chương này dùng chung một tiêu chí: số vòng lặp và thời gian cần để đưa $f(w_k) - f^*$ xuống dưới 10^{-6} . Mỗi cấu hình chạy ba lần và lấy trung vị thời gian.

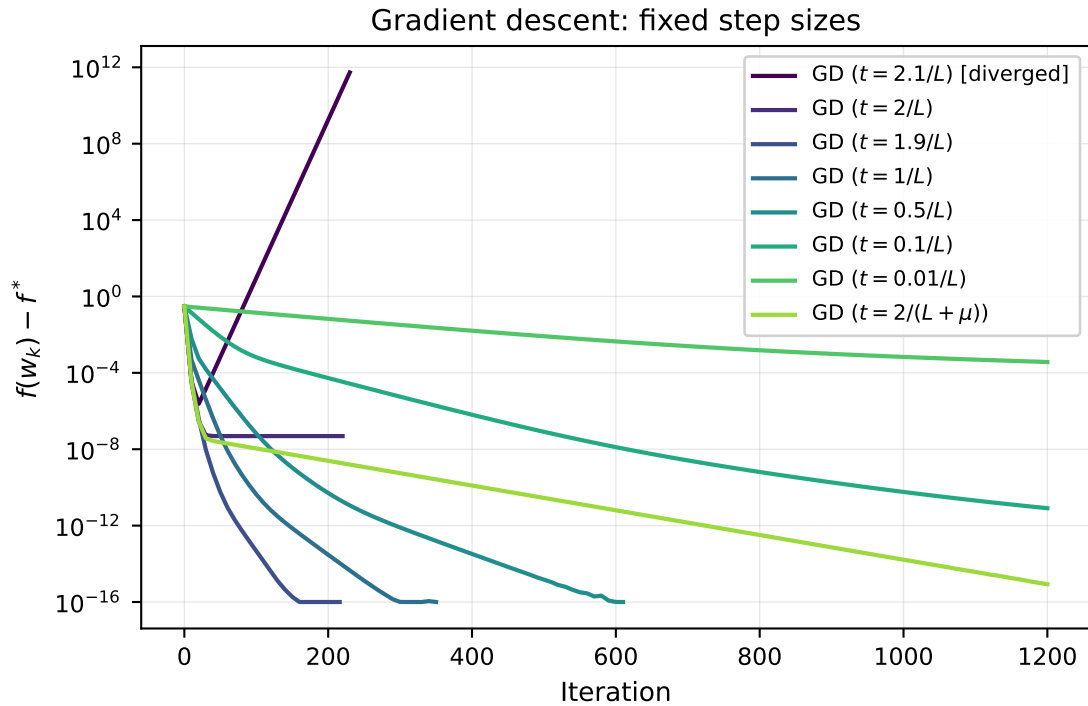
Mọi số đo thời gian trong báo cáo lấy trên cùng một máy, chip Apple M1 Pro với 16 GB bộ nhớ chạy macOS 26.5, Python 3.14.4, NumPy 2.5.1 liên kết với thư viện đại số tuyến tính Accelerate, SciPy 1.18.0 và scikit-learn 1.9.0. Vì đối máy thì giá trị tuyệt đối của các con số này đổi theo trong khi tỉ lệ giữa các thuật toán gần như giữ nguyên, phần bình luận bên dưới bám vào tỉ lệ nhiều hơn vào số giây. Mọi thành phần ngẫu nhiên dùng chung seed 0, gồm việc lấy mẫu 200 000 bản ghi, phép chia train và test, và thứ tự lô của SGD.

5.1 Gradient descent với độ dài bước cố định

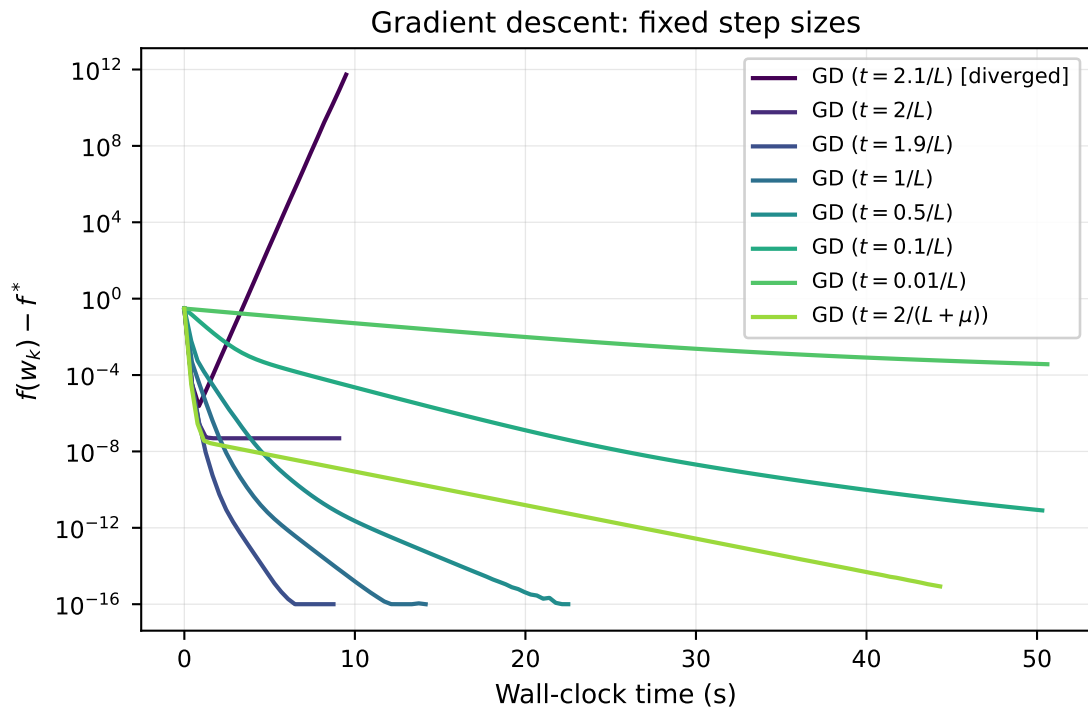
Với $t = 2.1/L$ hàm mục tiêu tăng tới $5,4 \times 10^{11}$, còn với $t = 1.9/L$ phương pháp hội tụ sau 20 vòng lặp, dù hai độ dài bước chỉ cách nhau 10% (bảng 5.1). Ngưỡng $2/L$ vì thế là một ranh giới thật: sai số của thành phần ứng với trị riêng λ_i của Hessian tắt theo hệ số $|1 - t\lambda_i|$ mỗi vòng lặp, nên hệ số của thành phần ứng với $\lambda_{\max} = L$ vượt quá 1 ngay khi t vượt $2/L$. Đúng tại ngưỡng, hệ số đó bằng 1 nên thành phần này đứng yên, khiến $t = 2/L$ dừng ở sai số $4,9 \times 10^{-8}$ thay vì đi tiếp tới giới hạn số học như các bước ngắn hơn.

Trong vùng ổn định, giảm t đi một nửa thì số vòng lặp tăng gấp đôi, thấy rõ ở dãy 20, 40, 80 vòng lặp ứng với $t = 1.9/L, 1/L, 0.5/L$ trên hình 5.1. Nên chọn t càng gần $2/L$ càng tốt miễn là còn an toàn, và giá trị lý thuyết $t = 2/(L + \mu)$ tuy tốt nhất trong lưới nhưng chỉ hơn $t = 1.9/L$ chừng 0,07 s, vì $\mu \ll L$ nên $2/(L + \mu) \approx 2/L$. Quan hệ tỉ lệ nghịch này chỉ đọc được khi ngân sách vòng lặp còn đủ: $t = 0.01/L$ hết 500 vòng vẫn dừng ở $3,7 \times 10^{-4}$, nên nó không cho một con số để đặt vào dãy trên.

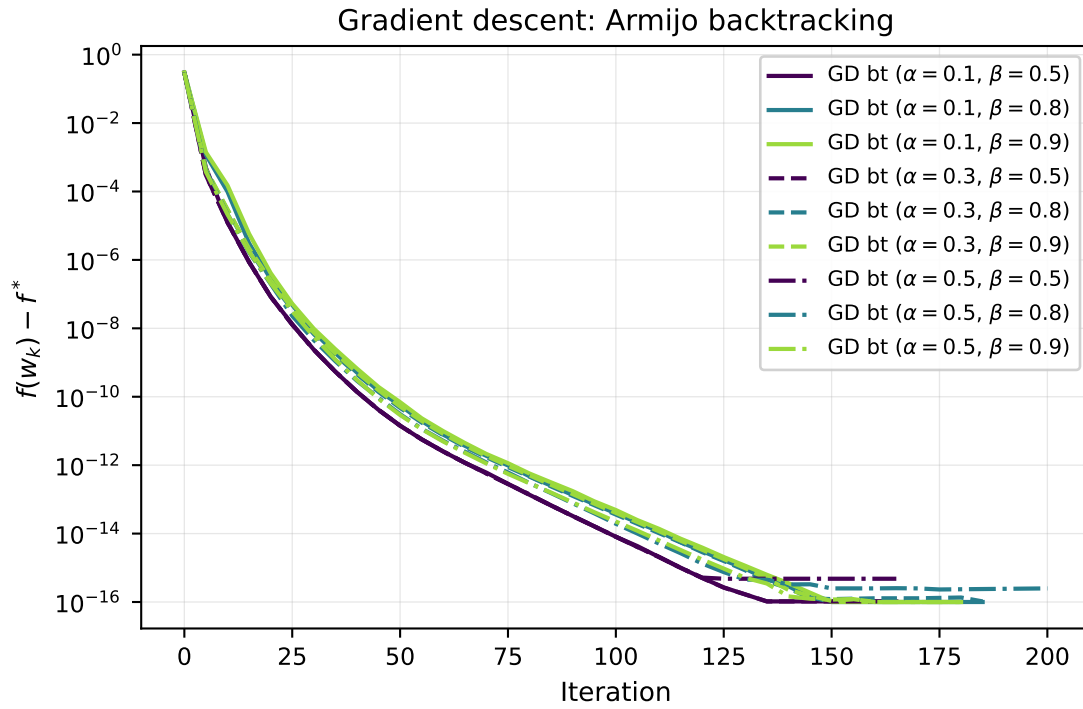
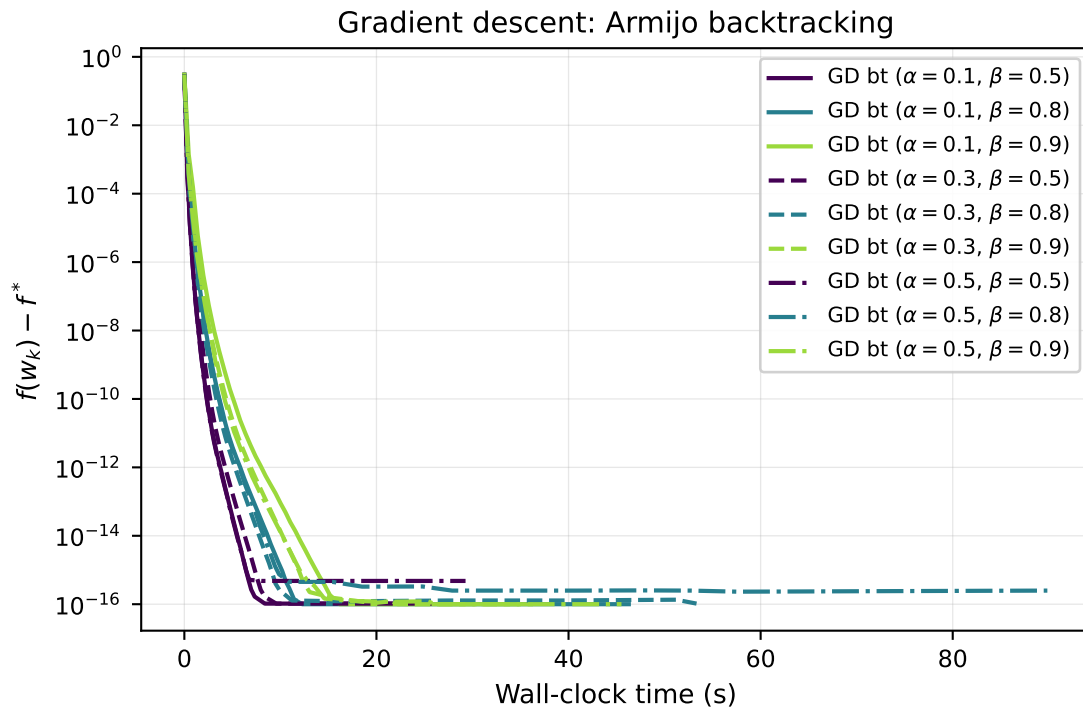
Thứ tự các đường giữ nguyên khi đổi trục hoành sang thời gian chạy (hình 5.2), chẳng hạn $t = 2/(L + \mu)$ mất 0,773 s so với 3,013 s của $t = 0.5/L$, đúng tỉ lệ giữa 20 và 80 vòng lặp. Mọi cấu hình ở đây gọi gradient đúng một lần mỗi vòng lặp và không có line search, nên thời gian chạy chỉ là số vòng lặp nhân với một hằng số. Hai trục sẽ tách nhau ngay khi mỗi vòng lặp tốn một số lần đánh giá hàm khác nhau, đúng tình huống của mục 5.2.



Hình 5.1: Gradient descent với các độ dài bước cố định, theo số vòng lặp.



Hình 5.2: Gradient descent với các độ dài bước cố định, theo thời gian chạy.

Hình 5.3: Ảnh hưởng của α và β , theo số vòng lặp.Hình 5.4: Ảnh hưởng của α và β , theo thời gian chạy.

Bảng 5.1: Gradient descent với độ dài bước cố định. Ngưỡng ổn định là $2/L = 0,745$.

Độ dài bước	Vòng lặp đạt 10^{-6}	Thời gian (s)	Trạng thái
$t = 2.1/L$	không đạt	–	phân kỳ
$t = 2/L$	20	0,858	dừng ở $4,9 \times 10^{-8}$
$t = 1.9/L$	20	0,844	hội tụ
$t = 2/(L + \mu)$	20	0,773	hội tụ
$t = 1/L$	40	1,650	hội tụ
$t = 0.5/L$	80	3,013	hội tụ
$t = 0.1/L$	380	15,926	hội tụ, chưa tới giới hạn số học
$t = 0.01/L$	không đạt	–	dừng ở $3,7 \times 10^{-4}$

Bảng 5.2: Chi phí của line search, đo trong 100 vòng lặp đầu. Ba cột giữa là số lần đánh giá hàm mục tiêu trên mỗi vòng lặp, ứng với ba giá trị của α .

	Đánh giá hàm / vòng lặp			Vòng lặp đạt 10^{-6}
	$\alpha = 0,1$	$\alpha = 0,3$	$\alpha = 0,5$	
$\beta = 0,5$	1,39	1,39	1,39	15
$\beta = 0,8$	2,46	2,47	2,37	20
$\beta = 0,9$	4,12	4,15	3,96	20

5.2 Gradient descent với backtracking

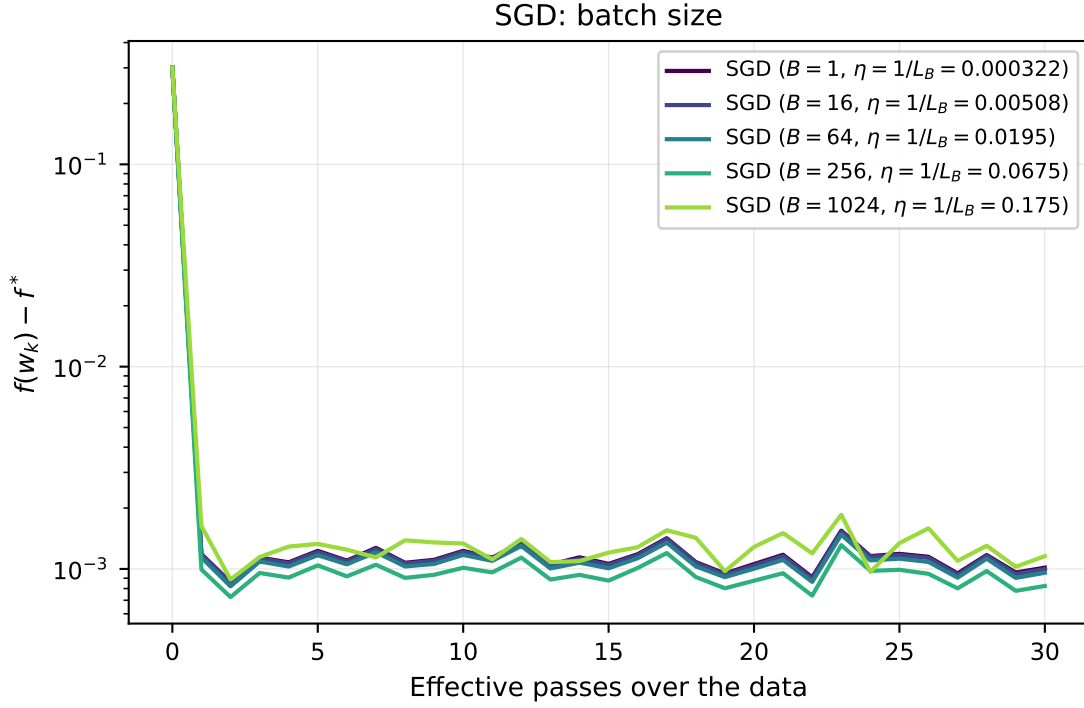
Bảng 5.2 đo số lần đánh giá hàm mục tiêu mà line search tiêu tốn trên mỗi vòng lặp, lấy trong 100 vòng lặp đầu. Phép đo chỉ có nghĩa trong giai đoạn thuật toán còn tiến triển, vì khi f đã chạm giới hạn số học thì không độ dài bước nào còn thỏa mãn điều kiện Armijo, line search tiêu hết ngân sách ở mọi vòng lặp và con số đo được tăng lên gấp năm lần.

Đọc bảng 5.2 theo hàng ngang, ba giá trị $\alpha \in \{0.1, 0.3, 0.5\}$ cho số lần đánh giá hàm chênh nhau nhiều nhất 4,8%, và ở $\beta = 0.5$ thì cả ba trùng khớp tới hai chữ số thập phân. Với hàm bậc hai, điều kiện Armijo chấp nhận t khi $t \leq 2(1 - \alpha)\|g\|^2/(g^\top Hg)$, nên thay đổi α trong khoảng này chỉ dịch ngưỡng đi một chút, thường không đủ để đổi số lần thu nhỏ. Ngưỡng đó co về 0 khi α tiến tới 1, nên kết luận trên chỉ đúng với các giá trị α vừa phải như lưới đang xét.

Đọc cùng bảng theo cột dọc thì β mới là tham số quyết định: $\beta = 0.5$ tốn 1,39 lần đánh giá mỗi vòng lặp và đạt 10^{-6} sau 15 vòng, còn $\beta = 0.9$ tốn khoảng 4,1 lần và cần 20 vòng, vì β càng gần 1 thì mỗi lần thu nhỏ càng ít và số lần thử tăng theo.

Hai cấu hình $\beta = 0.8$ và $\beta = 0.9$ gần trùng nhau trên trục vòng lặp (hình 5.3) nhưng tách hẳn nhau trên trục thời gian (hình 5.4), do chênh lệch nằm ở số lần đánh giá hàm trong mỗi vòng lặp chứ không ở số vòng lặp.

So với bước cố định tốt nhất, backtracking cần 0,852 s thay vì 0,773 s, tức chậm hơn 10%, đổi lại nó không cần



Hình 5.5: SGD với các kích thước lô khác nhau, theo số lượt duyệt dữ liệu.

biết L . Trên bài toán này L tính được dễ dàng nên bước cố định là lựa chọn hợp lý hơn, nhưng nhận xét ấy đảo chiều ngay khi d lớn tới mức không tính được trị riêng lớn nhất của ma trận Gram.

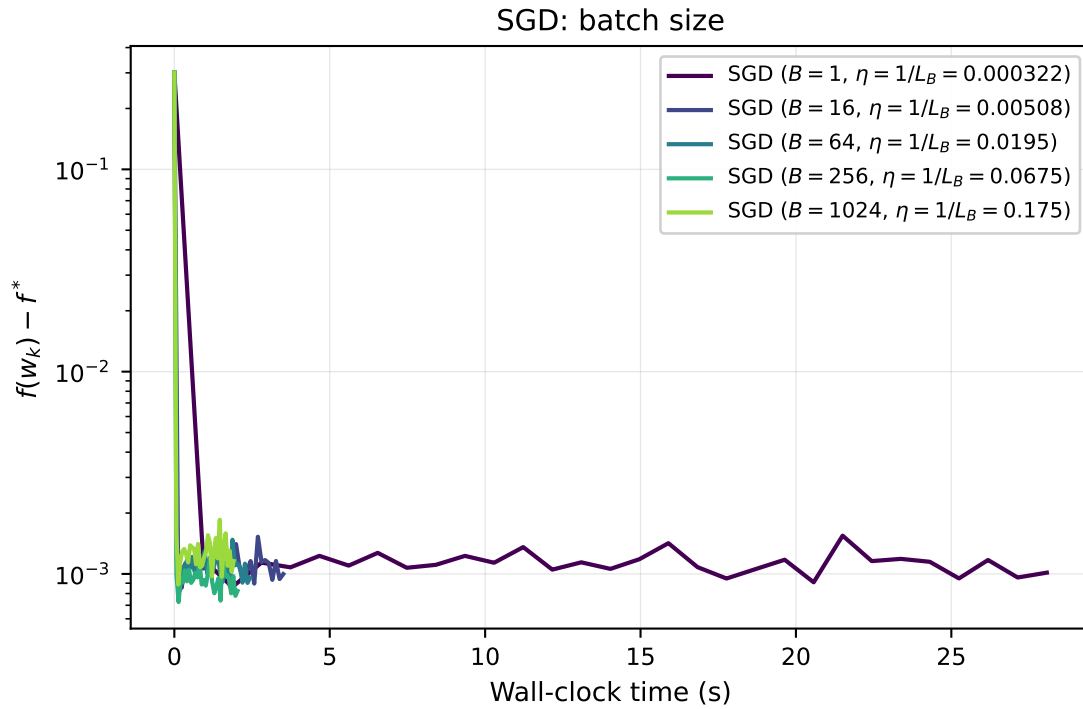
5.3 SGD: kích thước lô

Hằng số Lipschitz lớn nhất trong các mất mát một mẫu, tức $\max_i \|x_i\|^2 + \lambda$, đo được bằng 3109,7 và lớn hơn $L = 2,683$ đúng 1159 lần. Mỗi kích thước lô vì thế phải có độ dài bước riêng, do ngưỡng ổn định của một bước ngẫu nhiên phụ thuộc độ cong của các mẫu trong lô chứ không phụ thuộc L . Nhóm lấy độ dài bước $1/L_B$ với

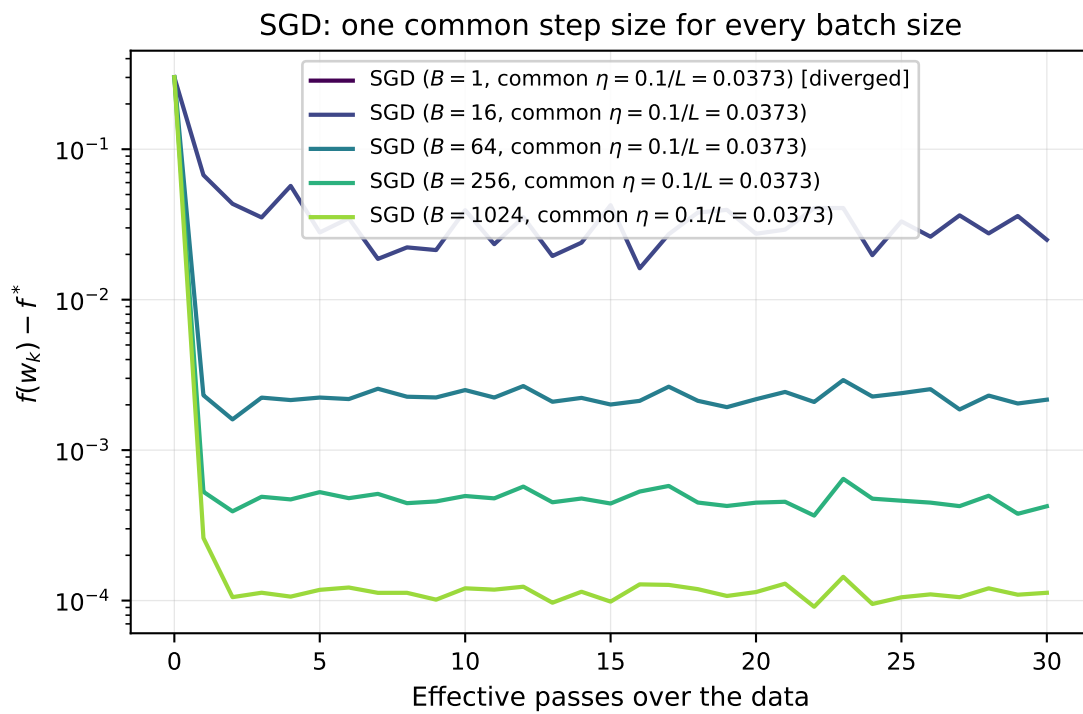
$$L_B = L + \frac{L_{\max} - L}{B}. \quad (5.1)$$

Thí nghiệm đối chứng ở hình 5.7 và 5.8 dùng chung một độ dài bước $0.1/L = 0,0373$ cho mọi kích thước lô, và kết quả tách hẳn hai đầu lưới: lô kích thước 1 phân kỳ thành NaN ngay trong lượt duyệt đầu tiên, còn lô kích thước 1024 đạt sai số $1,1 \times 10^{-4}$. Khoảng cách đó đúng như tỉ lệ 1159 lần giữa $L_{\max}$ và L dự báo qua (5.1), vì L_B tiến về L khi B tăng, nên chỉ các lô lớn mới nằm trong vùng ổn định của một độ dài bước tính theo L .

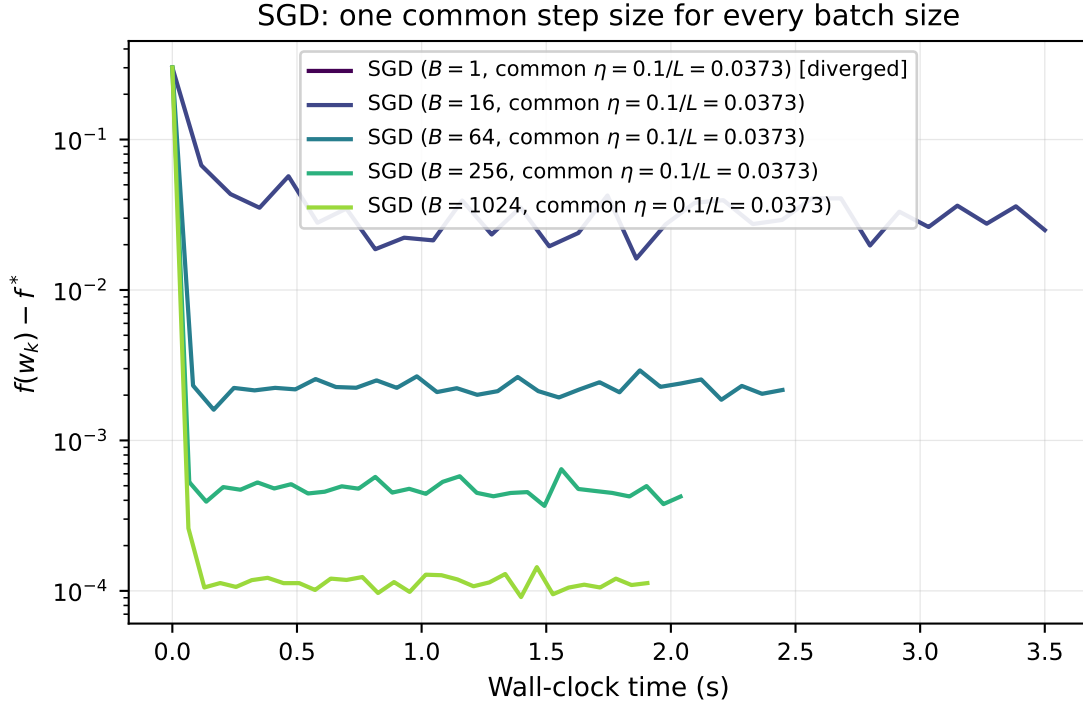
Khi mỗi kích thước lô nhận độ dài bước riêng, cả năm cấu hình đều về mức sai số 10^{-3} sau 30 lượt duyệt dữ liệu (hình 5.5), nhưng thời gian chạy chênh nhau 14 lần trên hình 5.6, 27,4 s với $B = 1$ so với 1,9 s với $B = 1024$. Chênh lệch đó đến từ chi phí cố định của mỗi lần cập nhật trong Python chứ không từ số phép tính, do $B = 1$



Hình 5.6: SGD với các kích thước lô khác nhau, theo thời gian chạy.



Hình 5.7: Cùng một độ dài bước $0.1/L$ cho mọi kích thước lô. Lô kích thước 1 phân kỳ.



Hình 5.8: Cùng một độ dài bước $0.1/L$, theo thời gian chạy.

Bảng 5.3: Sai số $f - f^*$ sau 30 lượt duyệt dữ liệu, $B = 64$.

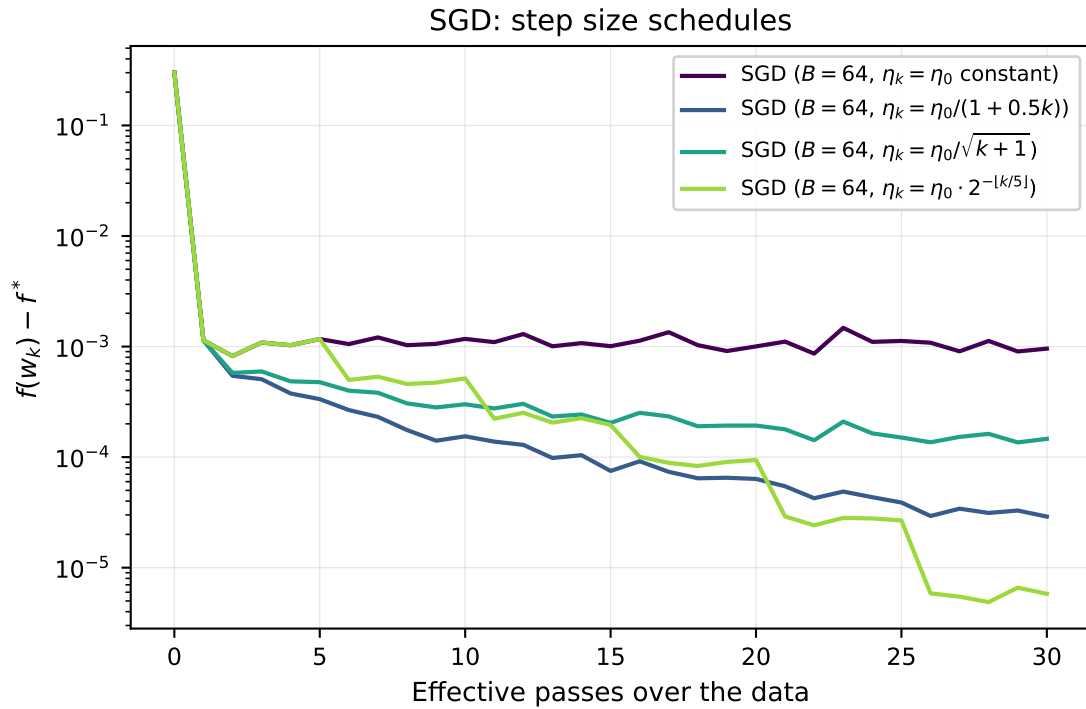
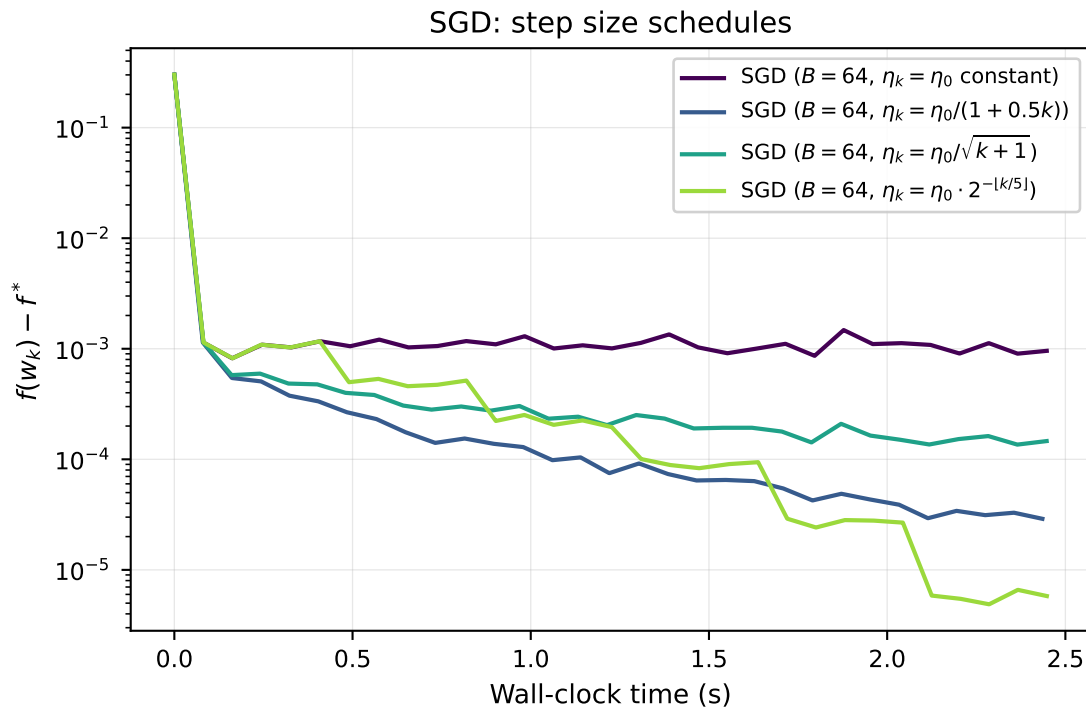
Quy tắc chọn η_k	Sai số cuối
$\eta_k = \eta_0$ (hằng)	$9,58 \times 10^{-4}$
$\eta_k = \eta_0 / \sqrt{k+1}$	$1,46 \times 10^{-4}$
$\eta_k = \eta_0 / (1 + 0.5k)$	$2,90 \times 10^{-5}$
$\eta_k = \eta_0 \cdot 2^{-\lfloor k/5 \rfloor}$	$5,80 \times 10^{-6}$

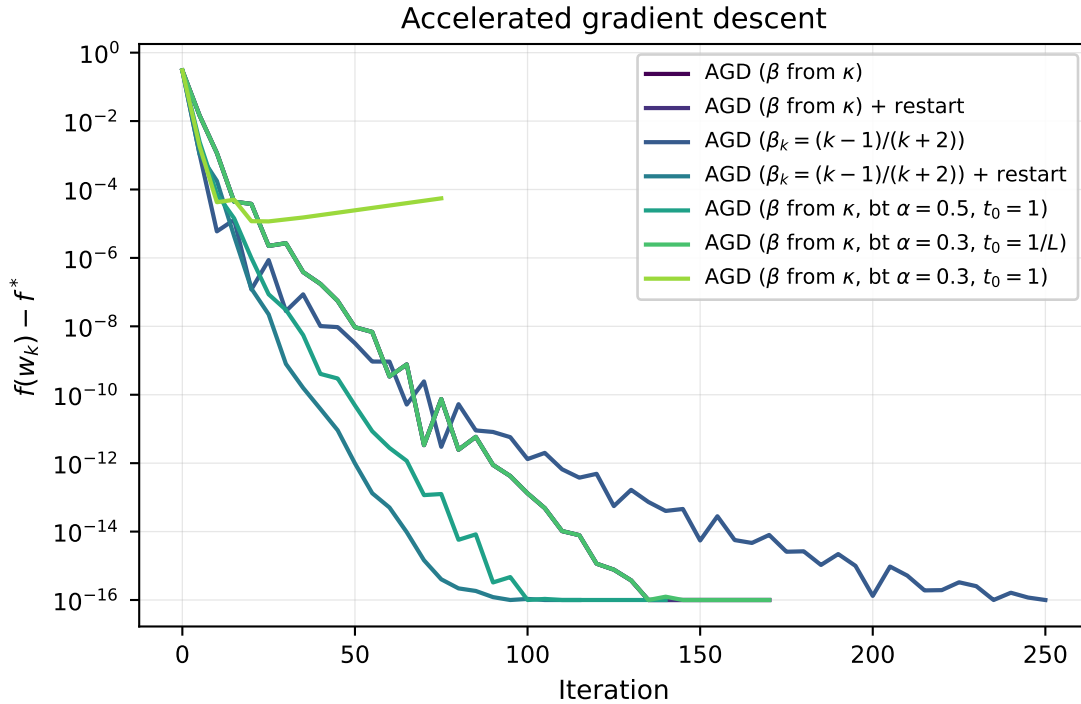
thực hiện 160 000 lần cập nhật mỗi lượt duyệt mà mỗi lần chỉ xử lý một dòng dữ liệu. Với một cài đặt bằng ngôn ngữ biên dịch, hoặc với d lớn hơn nhiều để phân số học lần ít phần điều phối, nhóm dự đoán tỉ lệ 14 lần này sẽ có lại đáng kể.

5.4 SGD: quy tắc chọn độ dài bước

Bốn quy tắc so sánh ở đây là bốn công thức tắt định cho η_k theo chỉ số vòng lặp ở (3.3), không phải biến thể của backtracking như ở mục 5.2. Cả bốn dùng chung $B = 64$ và chung ngân sách 30 lượt duyệt dữ liệu, nên cột sai số cuối của bảng 5.3 so sánh được trực tiếp với nhau.

Với độ dài bước hằng, SGD dừng ở sai số $9,58 \times 10^{-4}$ thay vì hội tụ về w^* , vì nó đi vào một lân cận bán kính cỡ $\mathcal{O}(\eta\sigma^2/\mu)$ rồi dao động quanh đó. Trên thang logarit theo lượt duyệt dữ liệu (hình 5.9), lân cận đó hiện ra thành một đường nằm ngang, đúng hành vi mà phép thử ở chương IV đòi hỏi ở một cài đặt chạy đúng. Bán kính

Hình 5.9: Các quy tắc chọn η_k , theo số lượt duyệt dữ liệu.Hình 5.10: Các quy tắc chọn η_k , theo thời gian chạy.



Hình 5.11: Các cấu hình momentum, có và không khởi động lại, theo số vòng lặp.

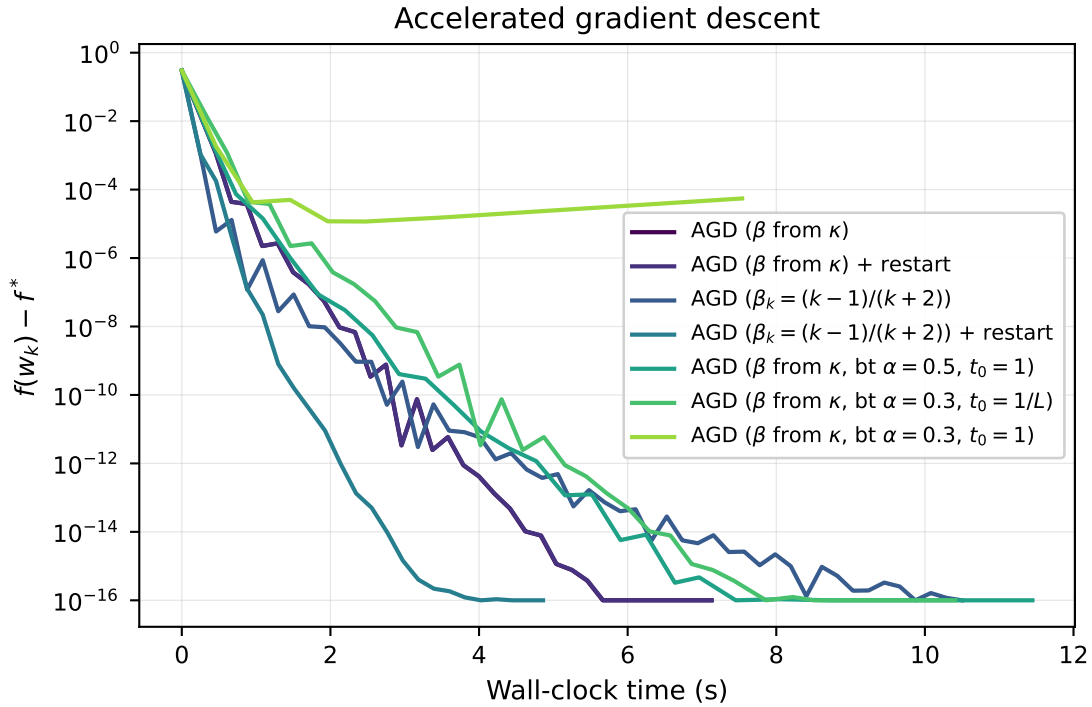
ấy chỉ co về không khi phương sai gradient bằng không, tức khi lô trùng toàn bộ tập huấn luyện.

Cả ba quy tắc có η_k giảm dần đều đưa sai số xuống thấp hơn, và mức cải thiện đi theo tốc độ giảm của η_k , từ $1,46 \times 10^{-4}$ với $1/\sqrt{k+1}$ tới $2,90 \times 10^{-5}$ với $1/(1+0.5k)$ rồi $5,80 \times 10^{-6}$ với quy tắc bậc thang, tức tốt hơn bước hằng 165 lần. Cơ chế vẫn là bán kính $\mathcal{O}(\eta\sigma^2/\mu)$ nói trên, lần này co lại cùng với η_k . Chi phí tính toán của cả bốn quy tắc như nhau, khoảng 2,4 s, nên bốn đường trên hình 5.10 kết thúc ở cùng một hoành độ và ở đây không có đánh đổi nào, bước hằng đơn thuần kém hơn.

Ngay cả quy tắc tốt nhất cũng chỉ xuống tới $5,80 \times 10^{-6}$ sau 30 lượt duyệt dữ liệu, chưa chạm ngưỡng 10^{-6} , nên trong ngân sách này SGD chỉ đáng dùng khi một nghiệm tạm với chi phí thấp là đủ. Ngân sách 30 lượt duyệt tương đương 30 lần quét toàn bộ dữ liệu, tức xấp xỉ chi phí của 30 vòng gradient toàn phần, và gradient descent chỉ cần 20 vòng để đạt đúng ngưỡng ấy; kết luận sẽ khác nếu n lớn tới mức một lần quét toàn bộ dữ liệu là quá đắt.

5.5 Accelerated gradient descent

Công thức momentum tăng dần $\beta_k = (k-1)/(k+2)$ đạt 10^{-6} sau 20 vòng lặp và 0,879 s, so với 35 vòng lặp và 1,496 s của công thức hằng tính từ κ (bảng 5.4, hình 5.11), dù về lý thuyết công thức hằng mới là công thức tối ưu cho hàm lồi mạnh. Nguyên do nằm ở phổ trị riêng của bài toán, vốn dồn về một vùng hẹp thay vì phủ kín đoạn $[\mu, L]$, nên bộ tham số thiết kế cho trường hợp xấu nhất mất lợi thế; mục 6.1 trình bày chi tiết. Trên



Hình 5.12: Các cấu hình momentum, có và không khởi động lại, theo thời gian chạy.

Bảng 5.4: Các cấu hình của phương pháp tăng tốc.

Cấu hình	Vòng lặp đạt 10^{-6}	Thời gian (s)	Tổng thời gian (s)
β hằng theo κ	35	1,496	7,12
β hằng, có khởi động lại	35	1,500	7,13
$\beta_k = (k-1)/(k+2)$	20	0,879	10,50
$\beta_k = (k-1)/(k+2)$, có khởi động lại	20	0,879	4,86
backtracking $\alpha = 0.5, t_0 = 1$	20	1,462	11,45
backtracking $\alpha = 0.3, t_0 = 1/L$	35	2,031	10,41
backtracking $\alpha = 0.3, t_0 = 1$	không đạt	—	7,54

một bài toán có phổ trải đều giữa μ và L , thứ tự hai công thức sẽ trở lại đúng như lý thuyết dự đoán.

Cơ chế khởi động lại không đổi số vòng lặp cần để đạt 10^{-6} , nhưng giảm hơn một nửa tổng thời gian chạy tới giới hạn số học, từ 10,5 s xuống 4,86 s, vì nó khử được dao động ở giai đoạn cuối. Trên hình 5.12, hai đường chỉ tách nhau ở phần sau ngưỡng 10^{-6} còn trước đó chúng trùng nhau, tức lợi ích của khởi động lại nằm trọn ở phần đuôi. Ghép khởi động lại vào công thức β hằng thì gần như không đổi gì, đúng như dự đoán, vì công thức hằng không tạo ra dao động tuần hoàn để mà khử.

Backtracking cho phương pháp tăng tốc. Với $\alpha = 0.3$ và $t_0 = 1$, phương pháp không hội tụ mà dừng ở sai số $5,5 \times 10^{-5}$, tức điều kiện Armijo thông thường không đủ khi ghép với momentum.

Line search ở đây chạy tại điểm ngoại suy y_k chứ không tại w_k , nên nó chỉ bảo đảm $f(w_{k+1}) < f(y_k)$ và

Bảng 5.5: Các biến thể của phương pháp bậc hai.

Biến thể	Số vòng lặp	Thời gian (s)
Newton $t = 1$, phân rã lại mỗi vòng	1	0,078
Newton $t = 1$, dùng lại phân rã	1	0,076
Newton damped, backtracking	1	0,089
Newton-CG, $\text{cg_tol} = 10^{-2}$	5	2,07
Newton-CG, $\text{cg_tol} = 10^{-6}$	2	1,83

không bảo đảm tính đơn điệu theo w . Điều kiện Armijo lại chỉ giới hạn độ dài bước dọc theo hướng gradient hiện tại, nên khi gradient nằm theo hướng có độ cong nhỏ, nó chấp nhận t lớn tới $\|g\|^2/(g^\top Hg)$, có thể lên tới $1/\mu = 100$, trong khi ngưỡng ổn định toàn cục chỉ là $2/L = 0,745$.

Hai cách xử lý sau đều đưa phương pháp về trạng thái hội tụ. Cách thứ nhất là dùng $\alpha = 1/2$, khi đó điều kiện Armijo trở thành đúng bỏ đề giảm

$$f(y - t\nabla f(y)) \leq f(y) - \frac{t}{2}\|\nabla f(y)\|^2, \quad (5.2)$$

tức chính là bất đẳng thức mà chứng minh hội tụ của phương pháp tăng tốc dựa vào. Cách thứ hai là khởi tạo $t_0 = 1/L$, khi đó line search chỉ có thể thu nhỏ từ một giá trị vốn đã an toàn.

Ngoài ra, momentum phải tính theo độ dài bước mà line search thực sự chọn:

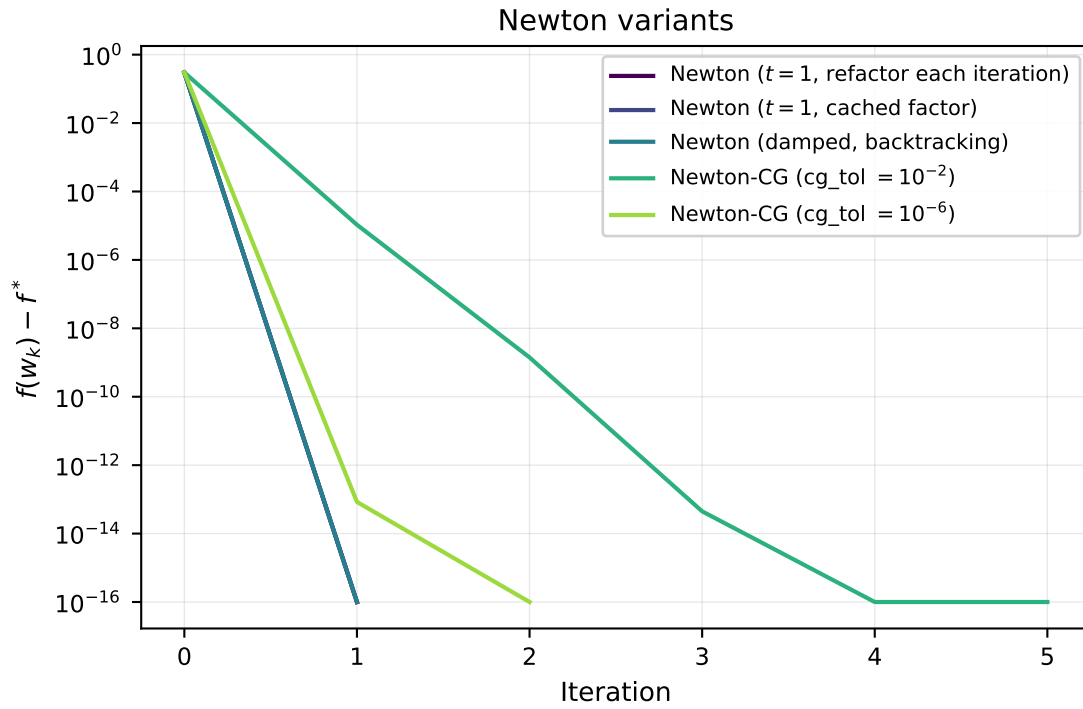
$$\beta = \frac{1 - \sqrt{t\mu}}{1 + \sqrt{t\mu}}. \quad (5.3)$$

Công thức quen thuộc $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ chỉ là trường hợp riêng khi $t = 1/L$. Trong lần cài đặt đầu, nhóm ghép độ dài bước do line search chọn với momentum tính theo $1/L$; hàm mục tiêu bùng lên $1,47 \times 10^4$.

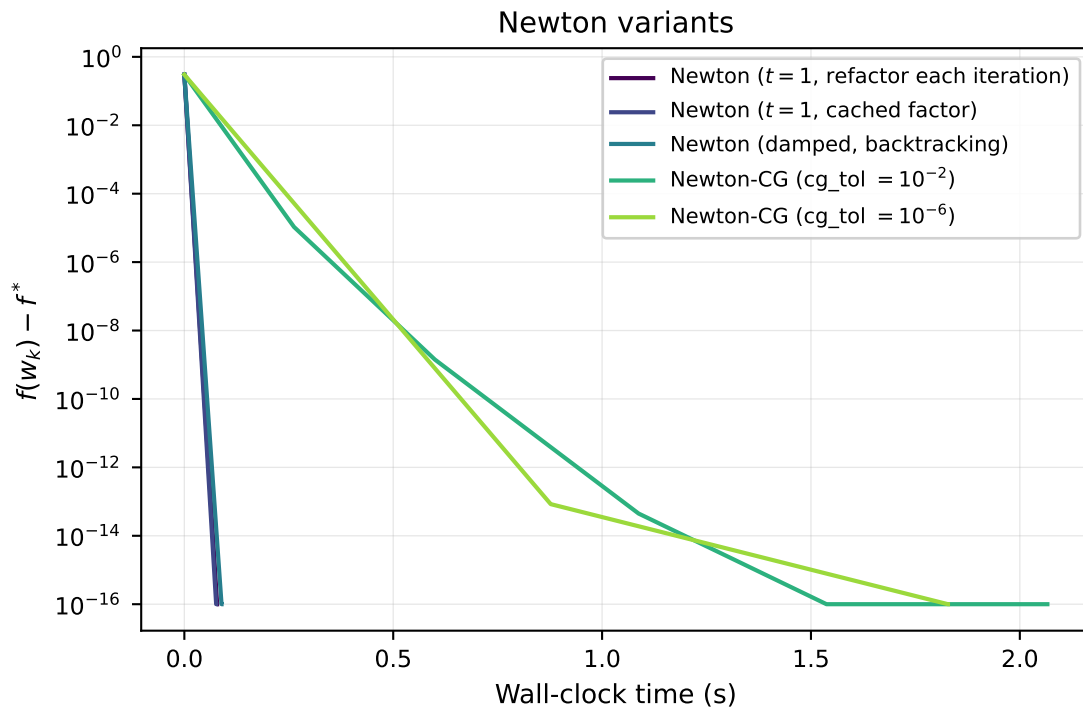
5.6 Phương pháp Newton

Bước Newton đầy đủ đưa nghiệm về đúng w^* sau một vòng lặp với sai số hàm mục tiêu bằng 0 trong số học dấu phẩy động (bảng 5.5, hình 5.13), đúng như (3.5) dự đoán. Backtracking chấp nhận ngay $t = 1$ và chỉ tốn một lần đánh giá hàm, nên biến thể damped mất 0,089 s so với 0,078 s của biến thể đầy đủ, tức hai biến thể không khác nhau về bản chất. Chúng chỉ tách nhau trên bài toán phi tuyến, nơi ở xa nghiệm thì $t = 1$ có thể bị từ chối.

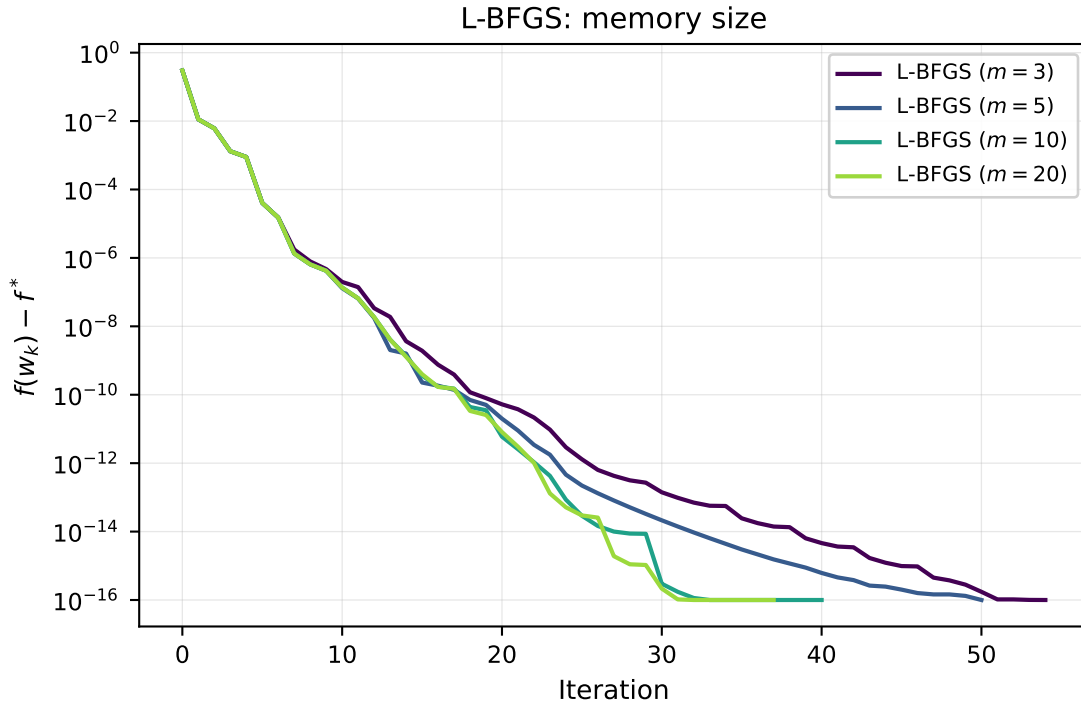
Cũng vì chỉ có một vòng lặp, phân rã lại Hessian hay dùng lại phân rã cũ không tạo ra khác biệt đo được, 0,078 s so với 0,076 s, do không có lần phân rã thứ hai nào để mà tiết kiệm. Hai biến thể này chỉ tách nhau khi Hessian



Hình 5.13: Các biến thể Newton, theo số vòng lặp.



Hình 5.14: Các biến thể Newton, theo thời gian chạy.



Hình 5.15: L-BFGS với các kích thước bộ nhớ m , theo số vòng lặp.

Bảng 5.6: L-BFGS với bốn kích thước bộ nhớ. Hai cột cuối đo tới khi thuật toán chạm giới hạn số học, không dừng ở ngưỡng 10^{-6} .

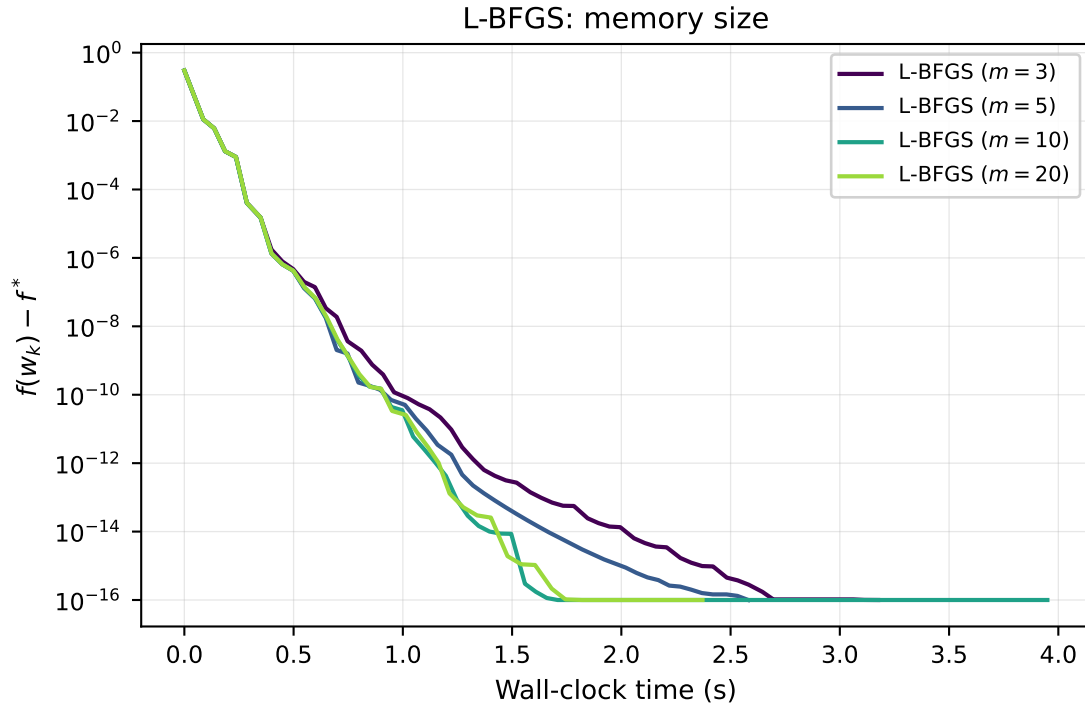
m	Vòng lặp đạt 10^{-6}	Thời gian (s)	Vòng lặp	Đánh giá hàm	Tổng thời gian (s)
3	8	0,448	54	143	3,18
5	8	0,448	50	105	2,58
10	8	0,448	40	227	3,95
20	8	0,449	37	112	2,37

phụ thuộc w và phân rã cũ không còn dùng lại được.

Trên trục thời gian ở hình 5.14, Newton-CG chậm hơn Newton 25 lần, vì với $d = 280$ thì chi phí $\mathcal{O}(d^3) \approx 2 \cdot 10^7$ của phân rã Cholesky nhỏ hơn nhiều so với chi phí một loạt tích Hessian nhân vector, mỗi tích tốn $\mathcal{O}(nd) \approx 4.5 \cdot 10^7$. Quan hệ này đảo khi d lớn tới mức không lập được ma trận $d \times d$, và đó cũng là tình huống duy nhất khiến Newton-CG đáng chọn.

5.7 L-BFGS với các kích thước bộ nhớ

Cả bốn kích thước bộ nhớ đạt ngưỡng 10^{-6} sau đúng 8 vòng lặp và 0,448 s (bảng 5.6), và trên hình 5.15 bốn đường chồng khít nhau cho tới quãng sai số 10^{-10} . Trong giai đoạn đầu, số cặp (s_k, y_k) tích lũy được còn ít hơn cả m nhỏ nhất trong lưới nên cả bốn cấu hình dựng lên gần như cùng một xấp xỉ nghịch đảo Hessian, và chọn m vì thế gần như không ảnh hưởng gì khi chỉ cần độ chính xác vừa phải.



Hình 5.16: L-BFGS với các kích thước bộ nhớ m , theo thời gian chạy.

Khác biệt chỉ hiện ra ở phần đuôi, nơi $m = 3$ cần 54 vòng lặp để chạm giới hạn số học còn $m = 20$ chỉ cần 37, vì càng giữ nhiều cặp (s_k, y_k) thì xấp xỉ nghịch đảo Hessian càng bao được nhiều hướng riêng của bài toán. Xu hướng này đều đặn theo m , từ 54 xuống 50, 40 rồi 37.

Tổng thời gian chạy lại không giảm đều theo m trên hình 5.16, vì $m = 10$ tốn 3,95 s trong khi $m = 3$ chỉ tốn 3,18 s dù cần nhiều hơn 14 vòng lặp. Chênh lệch đến từ line search, vốn tiêu 227 lần đánh giá hàm mục tiêu ở $m = 10$ so với 105 lần ở $m = 5$, trong khi chi phí $\mathcal{O}(md)$ của đệ quy hai vòng với $m \leq 20$ trên bài toán $d = 280$ nhỏ hơn hẳn chi phí một lần đánh giá f . Với d hoặc m lớn hơn nhiều thì phần $\mathcal{O}(md)$ mới chi phối và tổng thời gian sẽ tăng đều theo m .

Bảng so sánh tổng hợp lấy $m = 3$ vì nó nhanh nhất tới ngưỡng 10^{-6} , dù bốn cấu hình chỉ cách nhau khoảng một phần nghìn giây ở ngưỡng đó, tức nằm trong dao động của phép đo thời gian.

VI. So sánh tổng hợp

Phương pháp Newton. Toàn bộ lời giải mất 0,076 s (bảng 6.1), ít hơn 10 lần so với thời gian gradient descent cần chỉ để đạt 10^{-6} , và ít hơn 580 lần so với thời gian nó cần để chạm giới hạn số học. Hàm mục tiêu bậc hai nên bước Newton chính là nghiệm đóng, và khoảng cách này không nói được gì về các bài toán phi tuyến.

Các phương pháp bậc một ở độ chính xác vừa phải. Ba cấu hình GD, GD với backtracking và AGD đều đạt 10^{-6} trong khoảng 0,77 s đến 0,88 s, chênh nhau dưới 14%, và ba đường của chúng bám sát nhau suốt phần đầu hình 6.1. Giai đoạn này do các hướng có độ cong lớn chi phối, mà mọi phương pháp bậc một đều cắt các hướng ấy nhanh như nhau, nên nếu chỉ cần độ chính xác vừa phải thì chọn phương pháp nào cũng gần như không khác nhau.

Các phương pháp bậc một ở độ chính xác cao. Để đi tới giới hạn số học, AGD mất 4,86 s còn GD mất 44,3 s, tức chậm hơn 9 lần (hình 6.2). Lợi thế của gia tốc chỉ hiện ra ở phần đuôi, nơi μ chi phối tốc độ hội tụ chứ không còn là các hướng có độ cong lớn. Thứ hạng ở đây vì thế khác hẳn thứ hạng ở ngưỡng 10^{-6} , và đó là lý do mỗi so sánh cần cả biểu đồ theo vòng lặp lẫn biểu đồ theo thời gian.

6.1 Đối chiếu với tốc độ lý thuyết

Với hàm bậc hai lồi mạnh và độ dài bước $t = 2/(L + \mu)$,

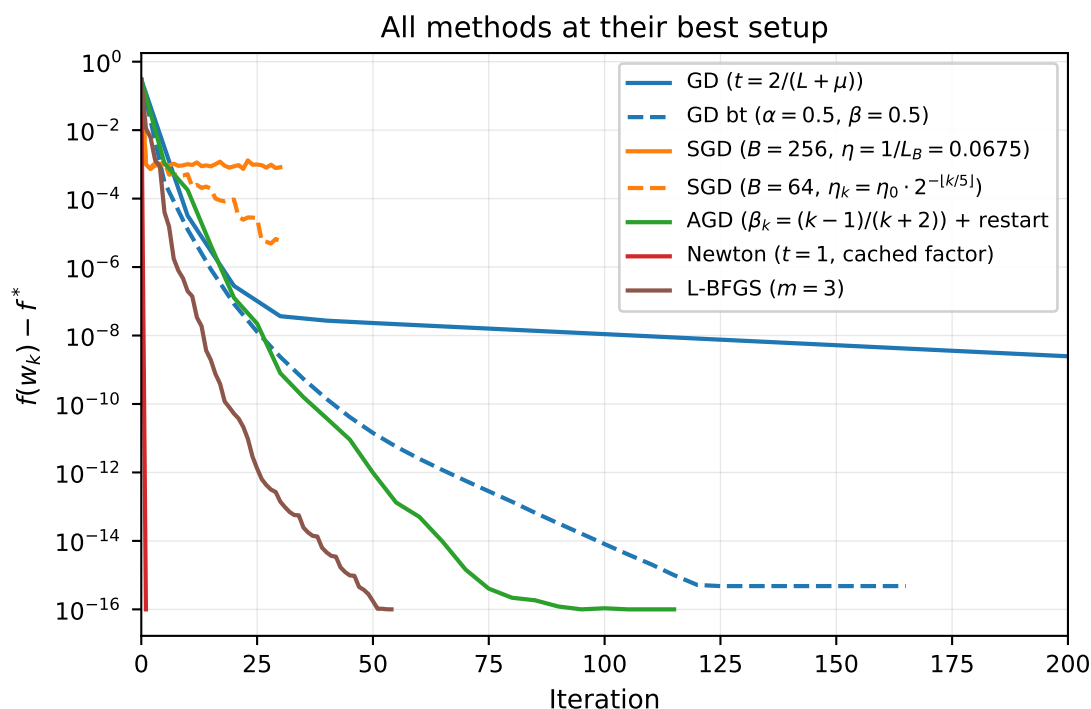
$$f(w_k) - f^* \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^{2k} (f(w_0) - f^*), \quad (6.1)$$

còn phương pháp tăng tốc có hệ số co rút cỡ $1 - \kappa^{-1/2}$.

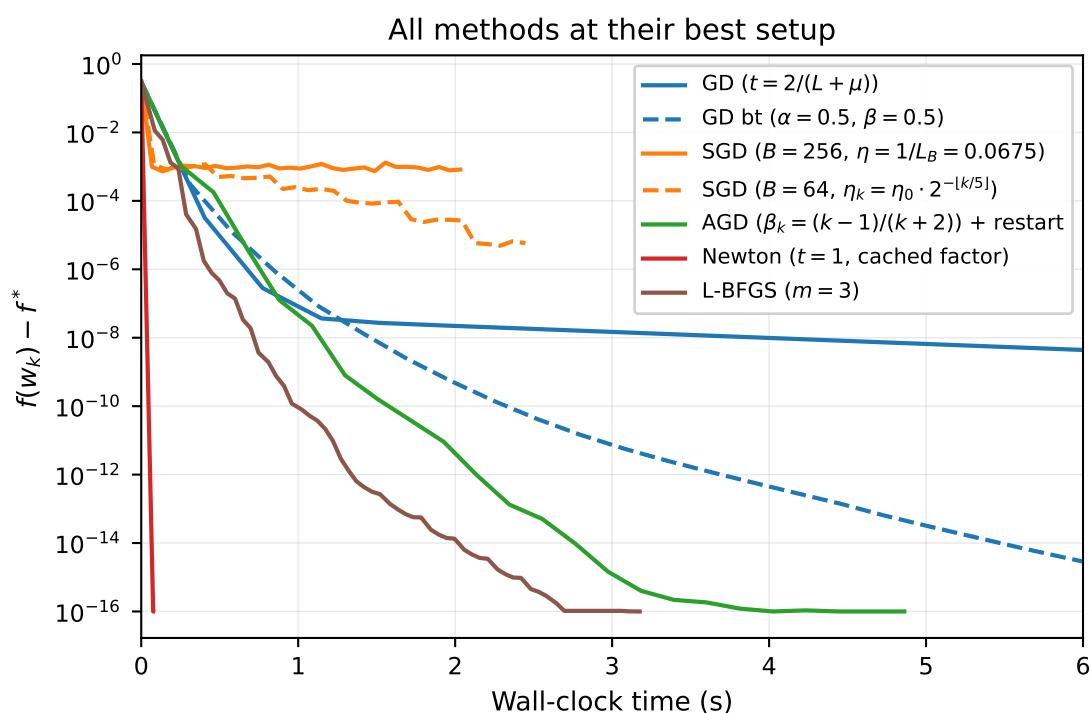
Nhóm ước lượng hệ số co rút thực nghiệm bằng cách khớp đường thẳng vào phần tuyến tính ở đuôi của đồ thị $\log_{10}(f(w_k) - f^*)$ theo k , rồi đối chiếu với cận lý thuyết trong bảng 6.2.

Với gradient descent, hệ số quan sát được là 0,985 20 và trùng với cận lý thuyết tới năm chữ số thập phân (bảng 6.2, hình 6.3). Cận (6.1) không chỉ đúng mà còn chặt, vì ở giai đoạn đuôi đúng thành phần ứng với trị riêng nhỏ nhất chi phối sai số, và tốc độ tắt của thành phần đó chính là $(\kappa - 1)/(\kappa + 1)$.

Với phương pháp tăng tốc, hệ số quan sát được là 0,654 56 so với cận 0,938 95, tức tốt hơn hẳn cận, nhưng hai con số vẫn không mâu thuẫn nhau. Cận lý thuyết là cận trên cho trường hợp xấu nhất trong lớp hàm lồi mạnh



Hình 6.1: So sánh các thuật toán ở cấu hình tốt nhất, theo số vòng lặp. Trục hoành cắt ở 200 vòng lặp cho dễ đọc; gradient descent với bước cố định chạy tiếp theo đúng đường thẳng đó tới vòng lặp 1200.



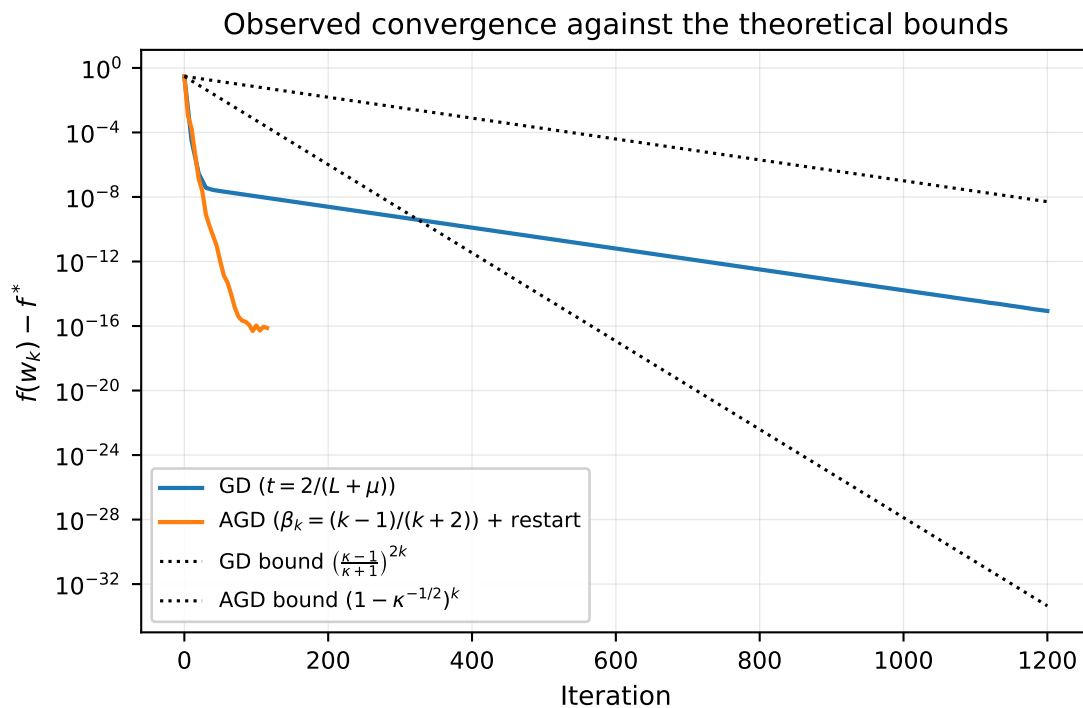
Hình 6.2: So sánh các thuật toán ở cấu hình tốt nhất, theo thời gian chạy. Trục hoành cắt ở 6 giây; gradient descent với bước cố định chạy tiếp tới 44,3s.

Bảng 6.1: Tổng hợp kết quả của từng thuật toán ở cấu hình tốt nhất. Cột *tổng thời gian* là thời gian chạy tới khi thuật toán không cải thiện được nữa, tức tới giới hạn số học.

Thuật toán	Cấu hình tốt nhất	Vòng lặp đạt 10^{-6}	Thời gian (s)	Tổng thời gian (s)
Newton	$t = 1$, Cholesky	1	0,076	0,076
L-BFGS	$m = 3$	8	0,448	3,18
GD	$t = 2/(L + \mu)$	20	0,773	44,32
GD	backtracking $\alpha = \beta = 0.5$	15	0,852	29,28
AGD	β_k tăng dần, có khởi động lại	20	0,879	4,86
SGD	$B = 64$, giảm bậc thang	không đạt	–	2,45

Bảng 6.2: Hệ số co rút mỗi vòng lặp, quan sát được so với cận lý thuyết.

Phương pháp	Quan sát	Cận lý thuyết
GD, $t = 2/(L + \mu)$	0,985 20	0,985 20
AGD, β_k tăng dần + khởi động lại	0,654 56	0,938 95



Hình 6.3: Đường hội tụ thực nghiệm đặt cạnh hai cận lý thuyết.

với κ cho trước, trong khi bài toán cụ thể ở đây có phổ trị riêng tập trung chứ không trải đều giữa μ và L . Cũng vì phổ tập trung mà công thức momentum tăng dần nhanh hơn công thức hằng tính từ κ , vốn được thiết kế cho trường hợp xấu nhất; trên một bài toán có phổ trải đều thì cả hai chênh lệch này đều thu hẹp lại.

Chênh lệch còn lớn hơn ở giai đoạn đầu: hai con số 20 và 40 vòng lặp để đạt 10^{-6} ở bảng 5.1 nhỏ hơn gần hai bậc so với ước lượng từ lý thuyết, vốn cho khoảng $\frac{1}{2}\kappa \ln(10^6) \approx 1850$ vòng lặp. Cận (6.1) vì thế mô tả đúng phần đuôi nhưng nói rất ít về giai đoạn đầu, và không dùng được để lập ngân sách vòng lặp cho bài toán này.

VII. So sánh với thư viện

Để phép so sánh có ý nghĩa, nhóm quy hàm mục tiêu của thư viện về cùng dạng với (1.1). Với `sklearn.linear_model.Ridge` [9], vốn cực tiểu hóa $\|Xw - y\|_2^2 + \alpha\|w\|_2^2$, quan hệ là $\alpha = \lambda n$. Bộ ước lượng `SGDRegressor` dùng `loss="squared_error"` và `penalty="l2"` thì có hàm mục tiêu trùng với (1.1) ngay khi $\alpha = \lambda$.

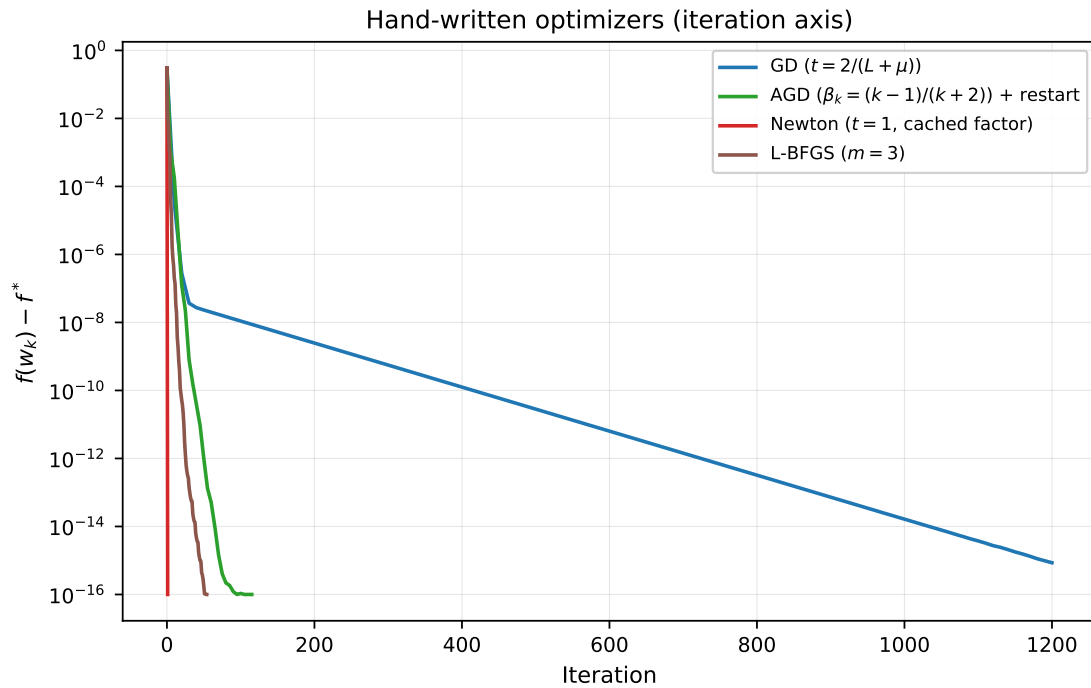
Với $\alpha = \lambda n = 1600$, nghiệm mà Ridge với solver `cholesky` trả về cách nghiệm đúng (1.3) một khoảng $1,06 \times 10^{-14}$, sai số tương đối $1,36 \times 10^{-14}$, và chênh lệch hàm mục tiêu bằng đúng 0 trong số học dấu phẩy động. Phép quy đổi vì thế kiểm chứng được bằng số chứ không chỉ bằng suy luận trên giấy. Vì mọi so sánh phía sau sẽ vô nghĩa nếu phép kiểm tra này thất bại, nhóm viết nó thành một khẳng định trong notebook thay vì một dòng in ra để đọc bằng mắt.

Các bộ giải trực tiếp của thư viện không có lịch sử vòng lặp, nên hình 7.1 chỉ đặt các phương pháp lặp tự cài đặt cạnh nhau, còn phép so sánh đầy đủ nằm ở hình 7.2.

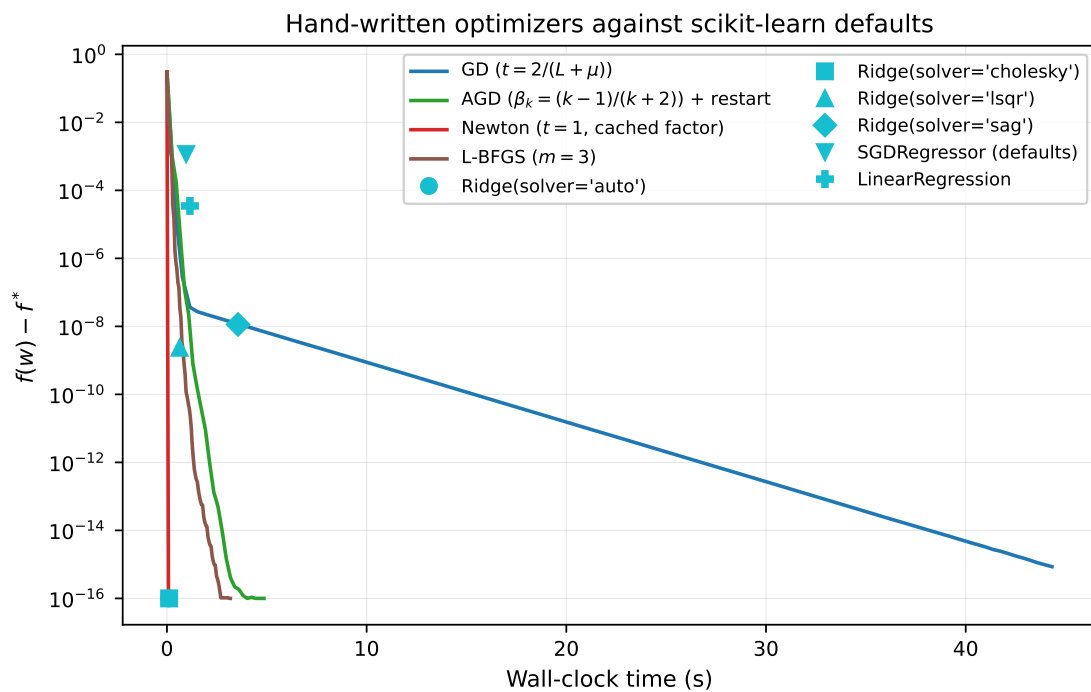
Ridge với solver trực tiếp đạt f^* với sai số cỡ sai số máy trong 0,11 s (bảng 7.1), hiện ra thành điểm nằm sát trục tung ở góc dưới bên trái hình 7.2. Solver ấy giải thẳng chính hệ phương trình mà một bước Newton giải, nên với bài toán có nghiệm đúng và d chỉ vài trăm thì khó có phương pháp lặp nào sánh được, còn phương pháp Newton tự cài đặt về bản chất làm đúng việc đó (chương III). Phương pháp lặp chỉ lấy lại chỗ đứng khi d lớn tới mức không phân rã được ma trận $d \times d$.

Ở tham số mặc định, `SGDRegressor` dùng lại cách f^* một khoảng $1,16 \times 10^{-3}$, cao gấp 200 lần sai số $5,80 \times 10^{-6}$ mà cấu hình SGD tự cài đặt đạt được sau khi điều chỉnh quy tắc chọn độ dài bước (bảng 5.3), và RMSE trên tập kiểm tra cũng kém hơn, 0,212 02 so với 0,206 01. Quy tắc chọn độ dài bước mặc định của bộ ước lượng này không tính theo L_B của bài toán, nên nó rơi vào đúng lân cận mà mục 5.4 mô tả. Nhận xét chỉ áp cho bộ ước lượng này ở tham số mặc định, chứ không áp cho thư viện nói chung.

RMSE trên tập kiểm tra của `LinearRegression` là 0,205 79, tốt hơn một chút so với mọi cấu hình dùng $\lambda = 0,01$, vì quy tắc một sai số chuẩn ở mục 2.3 cố ý chọn mức hiệu chỉnh mạnh hơn mức tối ưu về dự báo. Khoản đổi lấy là số điều kiện nhỏ hơn 100 lần, và nếu mục tiêu duy nhất là RMSE nhỏ nhất thì đánh đổi ấy không còn đứng vững.



Hình 7.1: So sánh các phương pháp lặp tự cài đặt, theo số vòng lặp.



Hình 7.2: So sánh mã tự viết với thư viện, theo thời gian chạy. Các cài đặt thư viện không có lịch sử vòng lặp nên hiện ra thành một điểm trên mặt phẳng (thời gian, sai số).

Bảng 7.1: Cài đặt mặc định của scikit-learn, đánh giá bằng hàm mục tiêu (1.1) của bài toán. Cột *sai số* là $f(\hat{w}) - f^*$.

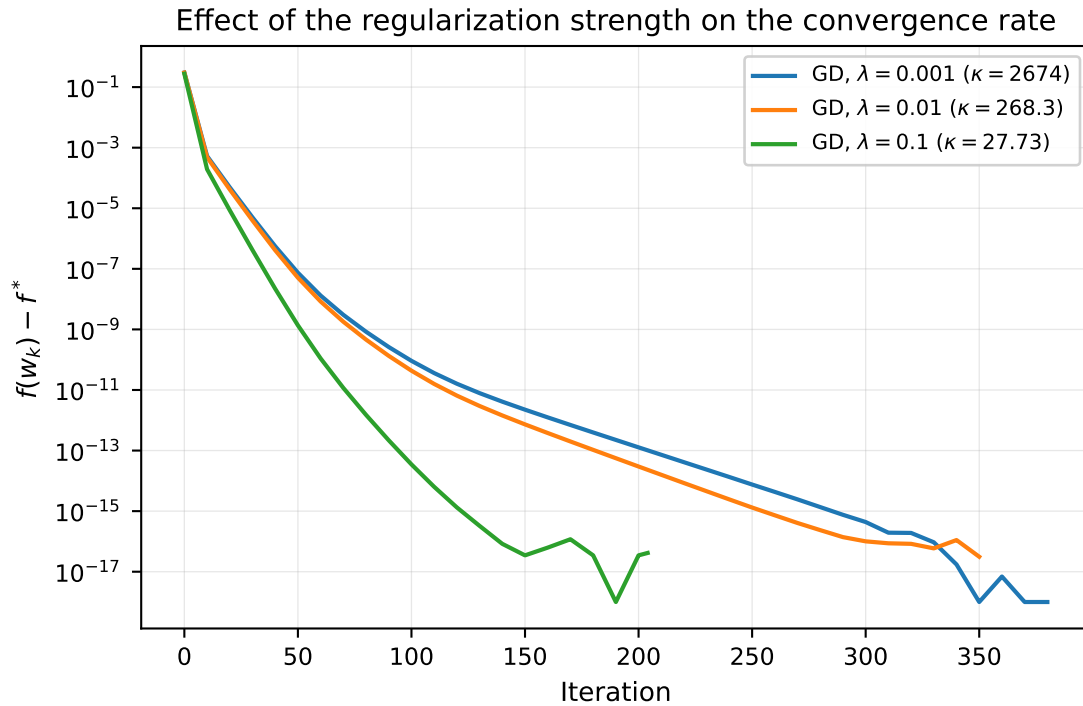
Bộ ước lượng	$f(\hat{w})$	sai số	thời gian (s)	RMSE test
Ridge(solver='auto')	0,023 743	0	0,113	0,206 01
Ridge(solver='cholesky')	0,023 743	0	0,112	0,206 01
Ridge(solver='lsqr')	0,023 743	$2,35 \times 10^{-9}$	0,621	0,206 01
Ridge(solver='sag')	0,023 743	$1,46 \times 10^{-8}$	3,955	0,206 01
SGDRegressor (mặc định)	0,024 906	$1,16 \times 10^{-3}$	1,018	0,212 02
LinearRegression	0,023 778	$3,48 \times 10^{-5}$	1,116	0,205 79

VIII. Ảnh hưởng của hệ số hiệu chỉnh

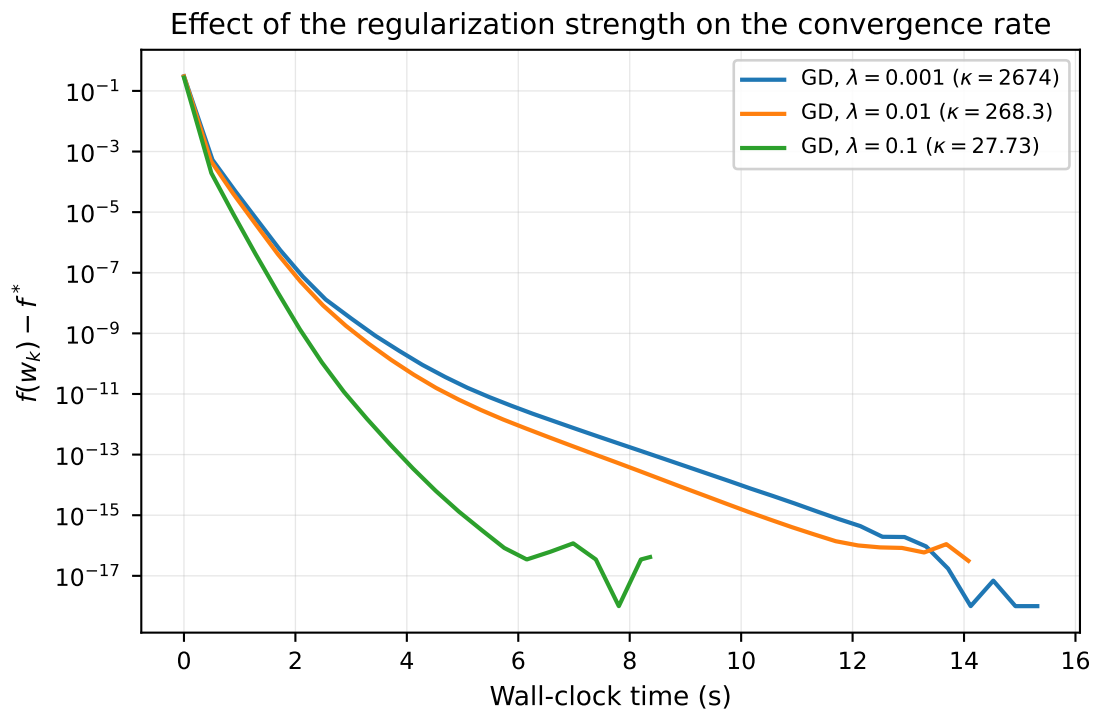
Ba dòng của bảng 8.1 cho κ bằng 2674, 268,3 và 27,73 ứng với λ bằng 0,001, 0,01 và 0,1, tức tăng λ lên 10 lần thì giảm κ đúng 10 lần, vì $\mu = \lambda$ trên bài toán này theo (1.5). Hiệu chỉnh Ridge do đó có hai vai trò tách biệt, một vai trò thống kê là chống quá khớp và một vai trò tối ưu hóa là cải thiện điều kiện của bài toán.

Hai vai trò này không cùng chiều, và mức ảnh hưởng của chúng lệch nhau rất xa. Từ $\lambda = 0,001$ lên 0,1, số điều kiện giảm 96 lần nhưng số vòng lặp cần thiết chỉ giảm từ 40 xuống 30 trên hình 8.1 và 8.2, trong khi RMSE trên tập kiểm tra xấu đi 6,5%. Số vòng lặp ít nhạy cảm như vậy vì cùng một lý do đã nói ở mục 6.1, rằng ở giai đoạn đầu các hướng có độ cong lớn mới chi phối gradient descent chứ không phải κ .

Nếu chỉ nhìn vào số vòng lặp cần cho độ chính xác vừa phải thì trên bài toán này, tăng λ để cải thiện điều kiện là một đánh đổi không có lợi. Nó chỉ đáng giá khi cần độ chính xác cao, nơi phần đuôi mới là phần tổn kém và κ lấy lại vai trò của mình.



Hình 8.1: Tốc độ hội tụ ứng với các giá trị λ khác nhau, theo số vòng lặp. Cả ba lần chạy dùng gradient descent với $t = 1/L$.



Hình 8.2: Tốc độ hội tụ ứng với các giá trị λ khác nhau, theo thời gian chạy.

Bảng 8.1: Hệ số hiệu chỉnh vừa quyết định số điều kiện, vừa quyết định chất lượng dự báo. Cả ba lần chạy dùng gradient descent với $t = 1/L$, nên cột số vòng lặp đặt cạnh được với dòng $t = 1/L$ của bảng 5.1.

λ	μ	κ	Vòng lặp đạt 10^{-6}	Thời gian (s)	RMSE test
0,001	0,001	2674	40	1,756	0,205 80
0,01	0,01	268,3	40	1,771	0,206 01
0,1	0,1	27,73	30	1,233	0,219 16

IX. Kết luận

Chọn độ dài bước. Chênh 10% qua ngưỡng $2/L$ đã đủ để chuyển từ hội tụ sang phân kỳ, với hàm mục tiêu tăng tới $5,4 \times 10^{11}$. Trong vùng ổn định, số vòng lặp tỉ lệ nghịch với độ dài bước nên chọn t càng gần $2/L$ càng tốt, còn giá trị lý thuyết $2/(L + \mu)$ tuy tốt nhất trong lưới nhưng gần như trùng với $2/L$ khi $\mu \ll L$.

Backtracking. Chi phí của line search do β quyết định chứ không do α , với $\beta = 0.5$ tốn 1,39 lần đánh giá hàm mỗi vòng lặp còn $\beta = 0.9$ tốn khoảng 4,1 lần. Trên bài toán này backtracking chậm hơn bước cố định tốt nhất 10%, đổi lại nó không cần biết L , nên nó chỉ đáng chọn khi L khó tính. Ghép với phương pháp tăng tốc thì điều kiện Armijo thông thường không đủ, phải lấy $\alpha = 1/2$ để điều kiện trùng với bổ đề giảm, hoặc phải khởi tạo $t_0 = 1/L$.

SGD. Độ dài bước phải theo kích thước lô, vì hằng số Lipschitz lớn nhất trong các mất mát một mẫu lớn hơn L tới 1159 lần trên bộ dữ liệu này. Giữ η_k cố định thì nghiệm dừng ở một lân cận quanh w^* , nên muốn sai số tiếp tục giảm thì phải hạ dần η_k ; trong bốn quy tắc đã thử, giảm theo bậc thang cho kết quả tốt nhất, hơn bước hằng 165 lần với cùng chi phí tính toán. Dù vậy SGD vẫn không đạt ngưỡng 10^{-6} trong ngân sách 30 lượt duyệt dữ liệu, tức nó chỉ có chỗ đứng khi một nghiệm tạm với chi phí thấp là đủ.

Phương pháp bậc hai. Với hàm mục tiêu bậc hai, Newton hội tụ sau một vòng lặp và bước Newton chính là nghiệm đóng, nên toàn bộ lời giải mất 0,076 s. L-BFGS đứng thứ hai với 0,448 s và là lựa chọn hợp lý hơn khi d lớn tới mức không lập được ma trận $d \times d$, còn Newton-CG chậm hơn Newton 25 lần ở $d = 280$ nên chỉ có lợi ở quy mô lớn hơn nhiều. Cả ba nhận xét này gắn với dạng bậc hai của hàm mục tiêu và không chuyển sang bài toán phi tuyến.

Khoảng cách giữa lý thuyết và thực nghiệm. Cận (6.1) mô tả đúng phần đuôi tới năm chữ số thập phân, nhưng nói rất ít về giai đoạn đầu, khi gradient descent đạt 10^{-6} sau 20 vòng lặp trong khi ước lượng từ lý thuyết là khoảng 1850. Khác biệt nằm ở chỗ phổ trị riêng của bài toán tập trung, không trải đều giữa μ và L như trường hợp xấu nhất mà cận nhắm tới. Cùng lý do đó khiến công thức momentum tăng dần nhanh hơn công thức hằng tính từ κ , dù về lý thuyết công thức hằng mới là tối ưu.

Tự cài đặt so với dùng thư viện. Trên bài toán có nghiệm đúng và d vài trăm, Ridge với solver trực tiếp đạt f^* với sai số cỡ sai số máy trong 0,11 s, nên nhóm tự cài đặt không nhằm chạy nhanh hơn thư viện. Nó cho thấy tham số mặc định của thư viện chưa chắc phù hợp với bài toán đang xét, như trường hợp SGDRegressor dùng cách f^* một khoảng $1,16 \times 10^{-3}$. Nó cũng cho biết mỗi thuật toán nhanh hoặc chậm ở giai đoạn nào và vì sao, thứ không đọc ra được từ một lần gọi hàm của thư viện.

Tài liệu tham khảo

- [1] Yurii Nesterov. *Lectures on Convex Optimization*. 2nd ed. Springer, 2018.
- [2] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd ed. Springer, 2006.
- [3] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. “Optimization Methods for Large-Scale Machine Learning”. In: *SIAM Review* 60.2 (2018), pp. 223–311.
- [4] Amir Beck and Marc Teboulle. “A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems”. In: *SIAM Journal on Imaging Sciences* 2.1 (2009), pp. 183–202.
- [5] Brendan O’Donoghue and Emmanuel Candès. “Adaptive Restart for Accelerated Gradient Schemes”. In: *Foundations of Computational Mathematics* 15.3 (2015), pp. 715–732.
- [6] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [7] Boris T. Polyak. “Some Methods of Speeding up the Convergence of Iteration Methods”. In: *USSR Computational Mathematics and Mathematical Physics* 4.5 (1964), pp. 1–17.
- [8] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations*. 2015.
- [9] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.